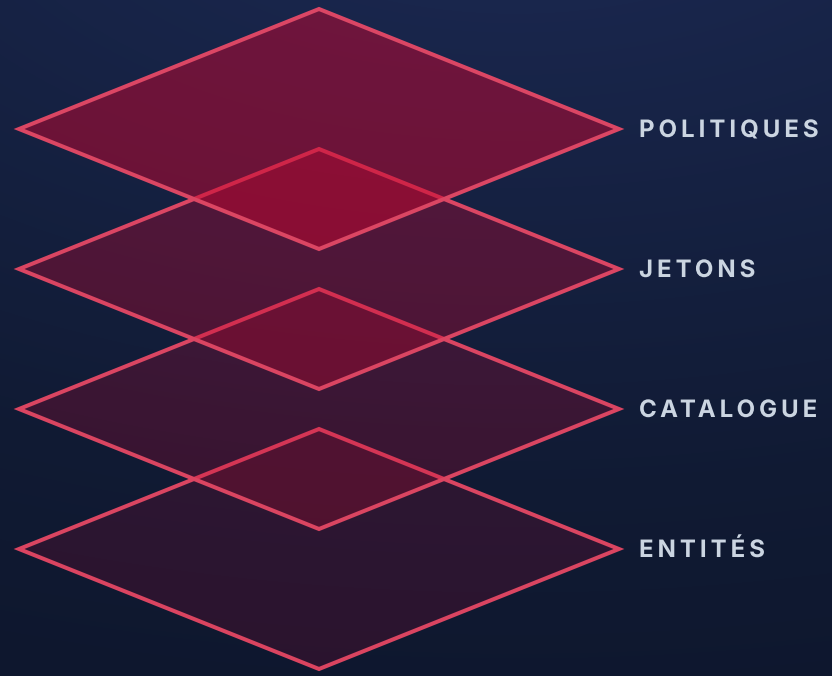
COMPRENDRE OPENSTACK · MODULE 8

Keystone en profondeur

Entités, catalogue de services, jetons d'authentification, sources d'identité, politiques d'accès — sur Canonical OpenStack (Sunbeam)

Domaines · projets · rôles | Catalogue | Fernet | LDAP · OIDC · SAML | RBAC



Version déployée : Canonical OpenStack 2026.1 LTS, donc OpenStack 2026.1 (les modules 1 à 5 suivent l'amont 2026.2) · documentations consultées en octobre 2026

INTRODUCTION

Objectif et déroulé

Objectif : comprendre comment Keystone répond aux quatre questions que pose toute requête dans OpenStack — qui es-tu, sur quoi agis-tu, où est le service, en as-tu le droit — et savoir le configurer et le diagnostiquer sur un cloud Sunbeam.

À l'issue de ce module, vous saurez :

- Décrire les **entités** de Keystone et leurs relations.
- Lire et exploiter le **catalogue de services**.
- Expliquer le cycle de vie d'un **jeton** : émission, contenu, validation, expiration, rotation des clés.
- Raccorder une **source d'identité** externe : annuaire LDAP ou fournisseur OIDC / SAML.
- Lire une **règle de politique** et prévoir qui peut faire quoi.
- Diagnostiquer un refus : **401**, **403**, service introuvable.

1 · Keystone dans le cloud
Rôle, architecture, déploiement sur Sunbeam, premiers pas.

2 · Les entités
Domaines, projets, utilisateurs, groupes, rôles, affectations, portées.

3 · Le catalogue de services
Services, points d'accès, régions, interfaces.

4 · Les jetons
Types, Fernet, clés, validation, autres moyens d'authentification.

5 · Les sources d'identité
Base locale, LDAP, fédération.

6 · Politiques d'accès
Règles, surcharges, portées, rôles par défaut.

7 · Exploiter Keystone sur Sunbeam
Diagnostic des refus, où regarder, disponibilité et sécurité.

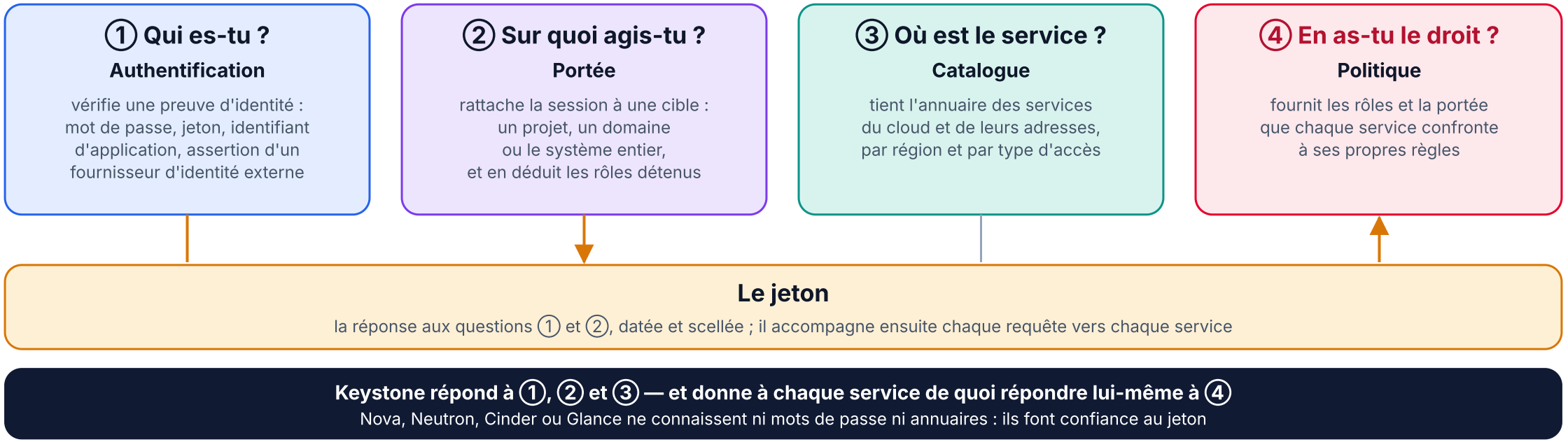
01

Keystone dans le cloud

Le premier service appelé par toute requête et le dernier que l'on peut se permettre de perdre : sans lui, plus personne ne s'authentifie et plus aucun service ne trouve les autres.

- Quatre questions, un service
- Architecture interne
- Premiers pas : s'authentifier
- Une requête de bout en bout
- Keystone sur Sunbeam

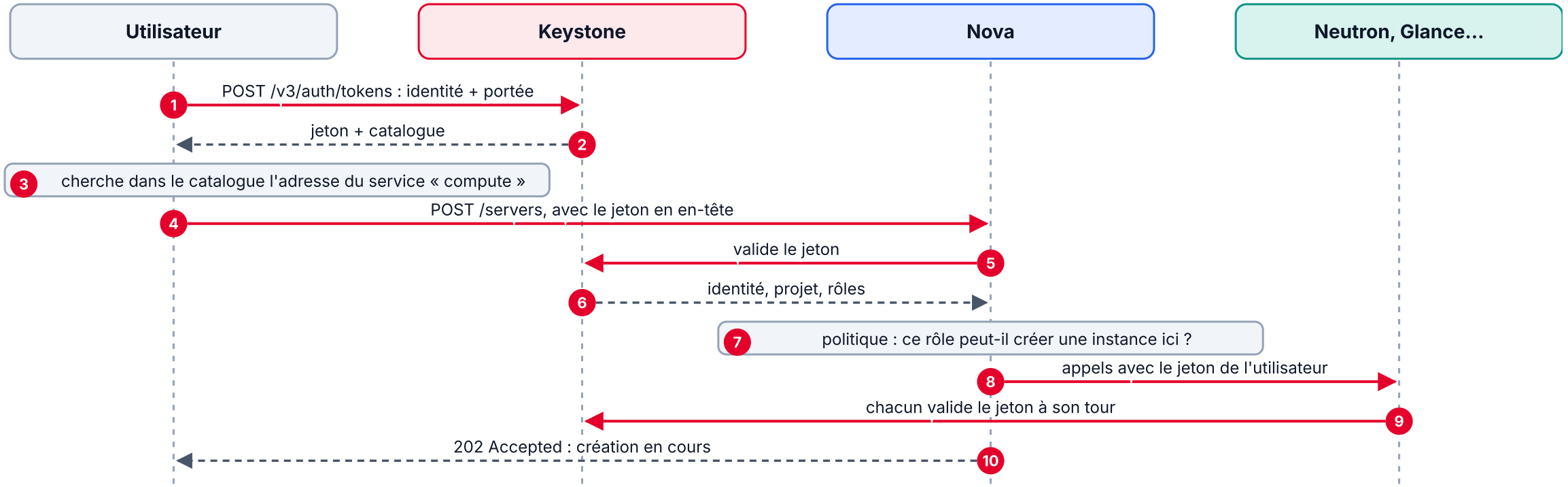
Quatre questions, un service



Ce que Keystone ne fait pas : il ne décide pas si l'on peut créer une instance ou supprimer un volume. Cette décision appartient à Nova ou à Cinder, à partir de ce que Keystone a certifié.

Version de l'API : une seule, la v3. Les notions de domaine, de groupe et de portée système n'existent que dans celle-ci.

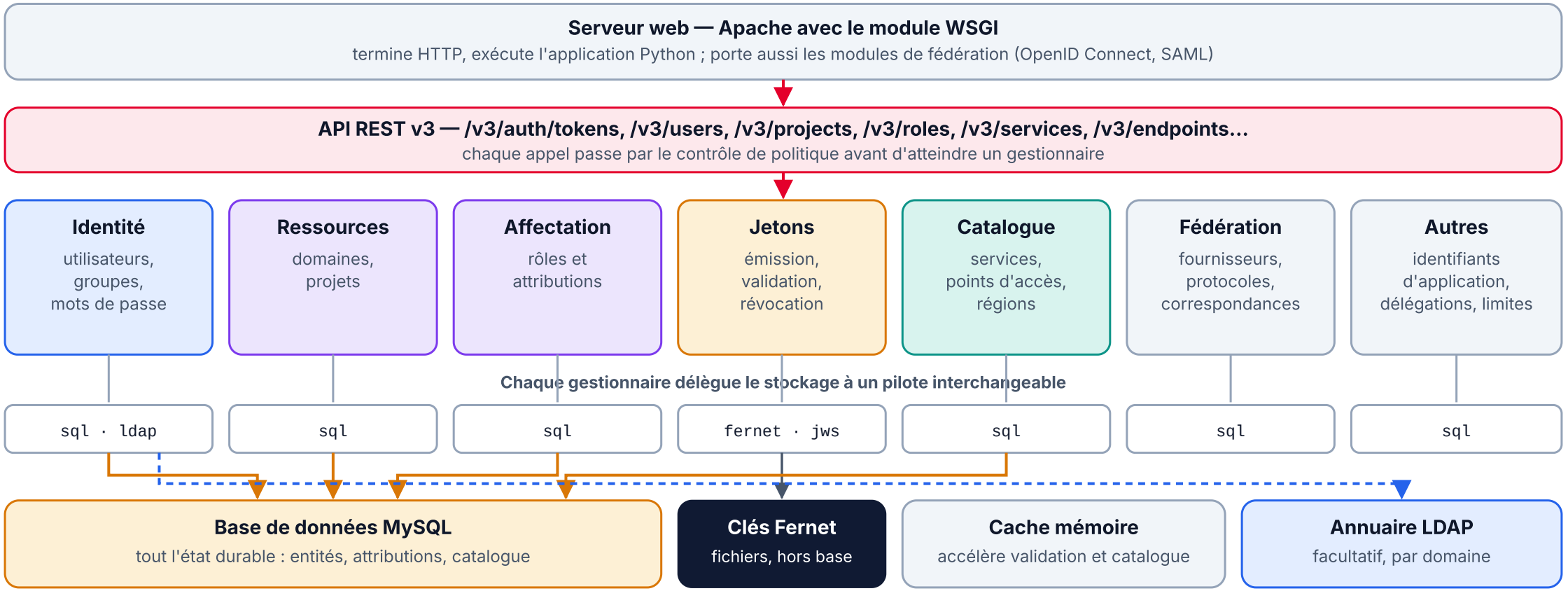
Une requête de bout en bout



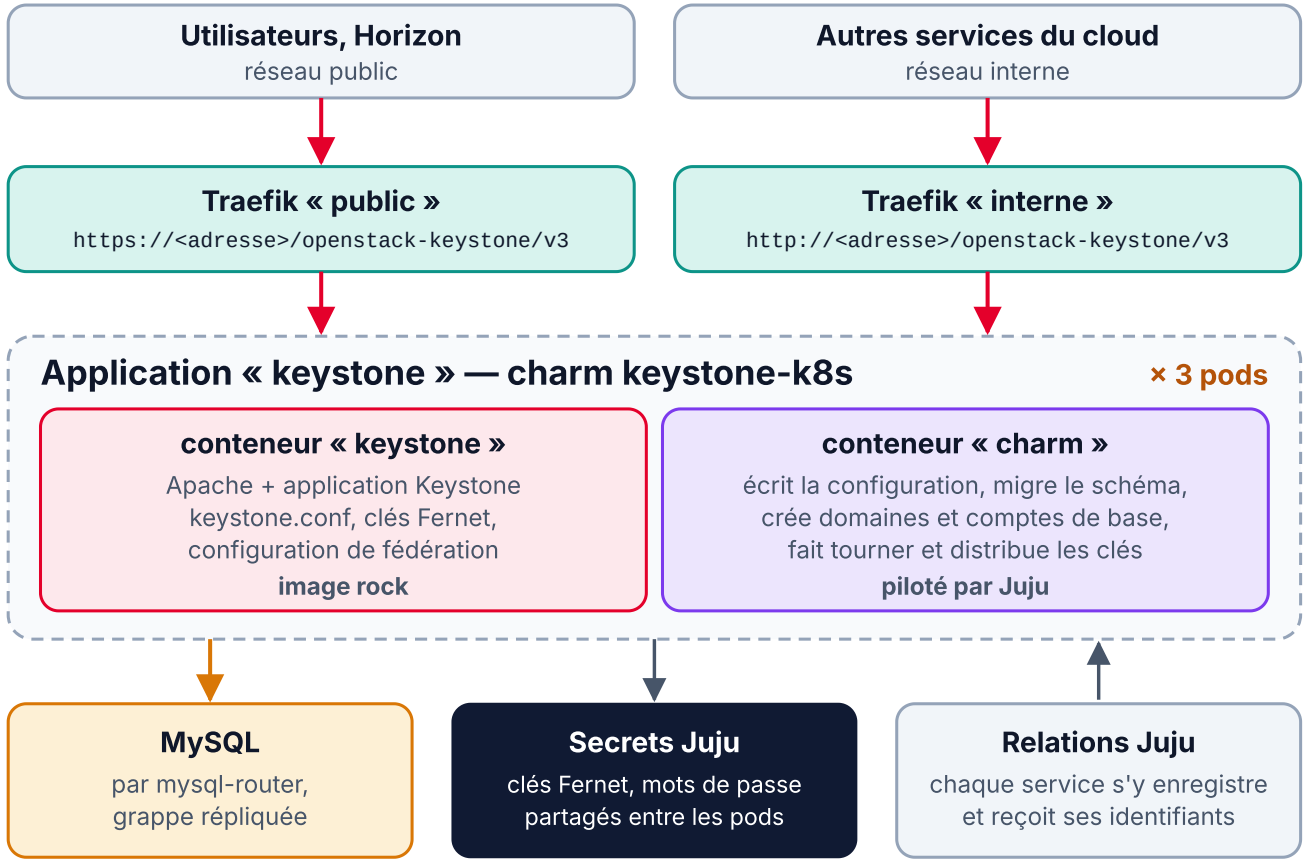
Deux appels à Keystone par opération : l'un par l'utilisateur, pour obtenir le jeton ; l'autre par le service, pour le valider. Le second est mis en cache par le service pour ne pas être répété à chaque requête.

Conséquence d'exploitation : si Keystone est lent, tout le cloud est lent ; s'il est indisponible, les jetons déjà validés et encore en cache fonctionnent quelques minutes, puis tout s'arrête.

Architecture interne



Keystone sur Sunbeam : où il tourne



- **Un pod par unité**, trois unités en haute disponibilité : n'importe laquelle traite n'importe quelle requête, car l'état est dans la base et les clés sont identiques partout.
- **Deux entrées :** les points d'accès public et interne passent par deux Traefik distincts, sous un préfixe de chemin.
- **Aucun fichier édité à la main :** `keystone.conf` est produit par le charm à partir de ses options et de ses relations.
- **Les autres services** n'ont pas de compte créé à la main : la relation avec Keystone crée leur utilisateur de service et inscrit leurs points d'accès.

```
juju status -m openstack keystone
juju config -m openstack keystone
juju actions -m openstack keystone
sudo k8s kubectl -n openstack get pods \
  -l app.kubernetes.io/name=keystone
```

> Options, actions et noms exacts varient selon la révision du charm : ces quatre commandes les montrent pour votre cloud.

Premiers pas : s'authentifier sur un cloud Sunbeam

```
# Identifiants de l'administrateur du cloud
sunbeam openrc > admin-openrc
source admin-openrc

# Identifiants de l'utilisateur de démonstration
sunbeam configure --openrc demo-openrc

# Vérifier : obtenir un jeton
openstack token issue

# Qui suis-je, avec quels rôles ?
openstack role assignment list \
  --user admin --names

# Que contient le cloud ?
openstack catalog list
```

Prérequis : le client `openstack` installé sur le poste, et l'adresse publique de Keystone joignable depuis ce poste. Avec TLS activé, le certificat de l'autorité doit être connu du client (`OS_CACERT`).

Variable	Rôle
`OS_AUTH_URL`	adresse publique de Keystone, se terminant par `/v3`
`OS_USERNAME` , `OS_PASSWORD`	la preuve d'identité
`OS_USER_DOMAIN_NAME`	le domaine où chercher l'utilisateur
`OS_PROJECT_NAME`	le projet visé : la **portée** demandée
`OS_PROJECT_DOMAIN_NAME`	le domaine de ce projet
`OS_REGION_NAME`	la région à choisir dans le catalogue
`OS_INTERFACE`	public, internal ou admin
`OS_IDENTITY_API_VERSION`	3

- **Un nom ne suffit jamais :** utilisateur et projet sont toujours désignés avec leur domaine, car deux domaines peuvent porter les mêmes noms.
- **Alternative au fichier de variables :** un fichier `clouds.yaml` et l'option `--os-cloud` , plus pratique pour jongler entre plusieurs comptes.
- **Ne pas laisser traîner** le fichier de l'administrateur : il contient un mot de passe en clair.

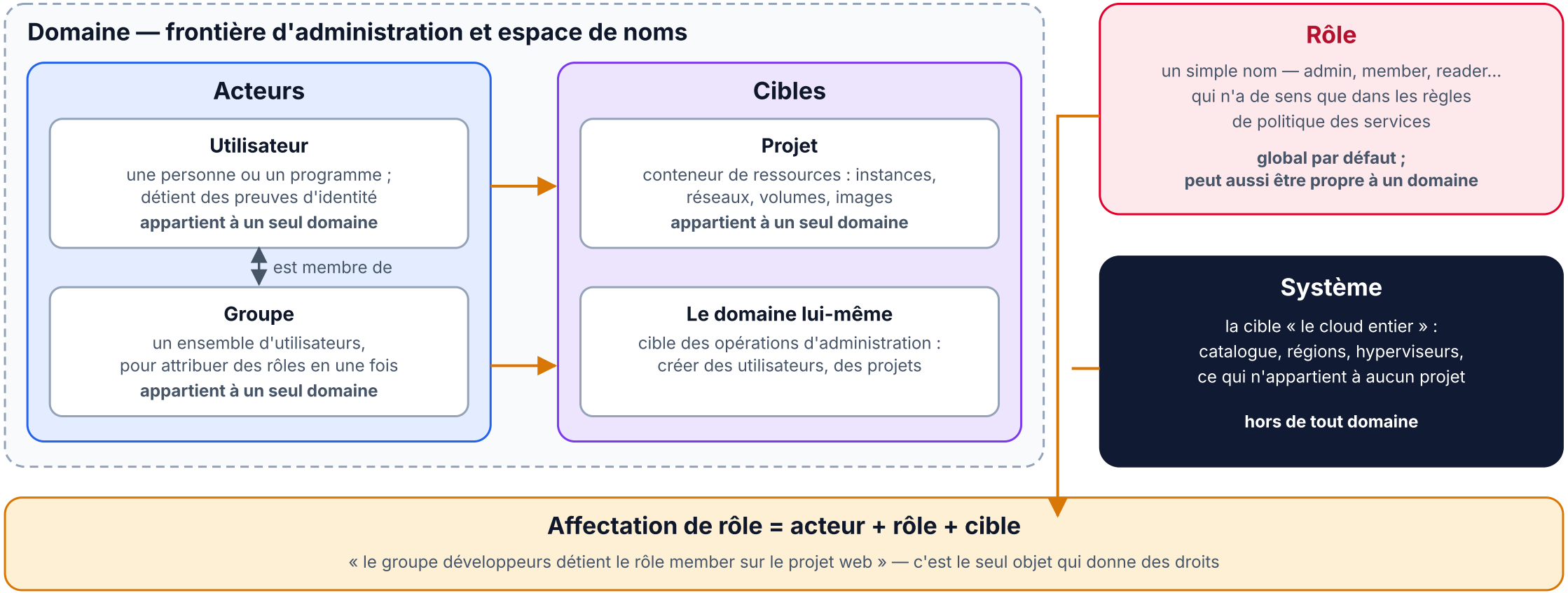
02

Les entités

Peu d'objets, mais des relations précises. Toute la gestion des droits d'OpenStack se ramène à une phrase : tel acteur détient tel rôle sur telle cible.

- Carte des entités
- Projets
- Rôles et rôles impliqués
- Les trois portées
- Ce que Sunbeam a créé pour vous
- Domaines
- Utilisateurs et groupes
- Affectations de rôles
- Les profils par défaut
- Commandes essentielles

Carte des entités



Domaines : la frontière d'administration

Ce qu'est un domaine

- **Un espace de noms :** les noms d'utilisateur, de groupe et de projet ne sont uniques qu'à l'intérieur d'un domaine. Deux domaines peuvent chacun avoir un projet « web ».
- **Une frontière de délégation :** on peut confier à quelqu'un l'administration d'un domaine — créer ses utilisateurs, ses projets, attribuer des rôles — sans lui donner la main sur le reste du cloud.
- **Un point de raccordement :** chaque domaine peut avoir sa propre source d'identité — la base locale pour l'un, l'annuaire de l'entreprise pour un autre.
- **La racine d'une hiérarchie de projets :** techniquement, un domaine est lui-même un projet particulier, sommet de son arbre.

> **Un domaine ne se supprime pas d'un geste :** il faut d'abord le désactiver. La suppression efface ses utilisateurs, groupes et projets dans Keystone, mais pas les ressources de ces projets dans les autres services.

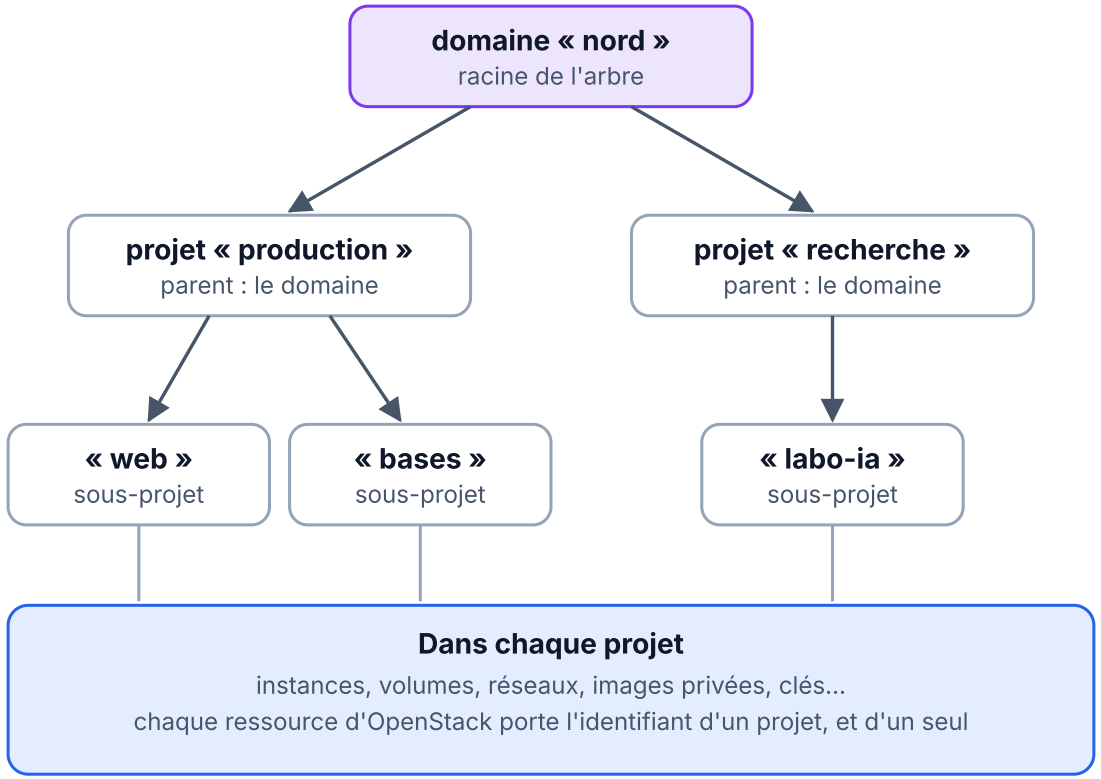
```
openstack domain list
openstack domain create --description "Filiale Nord" nord
openstack domain show nord

# Désactiver puis supprimer
openstack domain set --disable nord
openstack domain delete nord
```

> **Désactiver un domaine** coupe immédiatement l'authentification de tous ses utilisateurs et invalide leurs jetons : c'est l'interrupteur d'urgence pour un locataire.

 Approfondi au module 10

Projets : le conteneur des ressources



- **C'est l'unité de propriété :** quotas, facturation et isolation réseau se raisonnent par projet.
- **C'est la portée la plus courante** d'un jeton : on travaille « dans » un projet.
- **Hiérarchie :** un projet peut avoir des sous-projets ; un rôle peut être **hérité** vers le bas de l'arbre. L'isolation des ressources, elle, reste par projet.
- **Étiquettes** libres pour classer : environnement, centre de coût.
- **Désactivé**, un projet refuse toute nouvelle authentification vers lui, sans que ses ressources soient détruites.

```
openstack project create --domain nord production
openstack project create --domain nord \
  --parent production web
openstack project list --domain nord --long
openstack project set --tag prod web
```

> Hiérarchie et quotas font l'objet du module 10 ; on en retient ici le principe.

Utilisateurs et groupes

Utilisateur

- **Identifiant unique** dans tout le cloud ; **nom** unique dans son domaine seulement.
- **Trois origines possibles :** créé dans la base locale, lu dans un annuaire LDAP, ou créé à la volée lors d'une connexion fédérée.
- **Projet par défaut :** facultatif ; sert quand une authentification ne précise pas de portée.
- **Options par utilisateur :** dispense de changement de mot de passe, exemption de verrouillage, règles d'authentification multifacteur.

Groupe

- Contient des utilisateurs **du même domaine**.
- Un rôle attribué au groupe vaut pour tous ses membres, présents et futurs.
- Avec un annuaire ou une fédération, les groupes viennent de la source externe : on attribue les rôles aux groupes, **jamais** aux personnes.

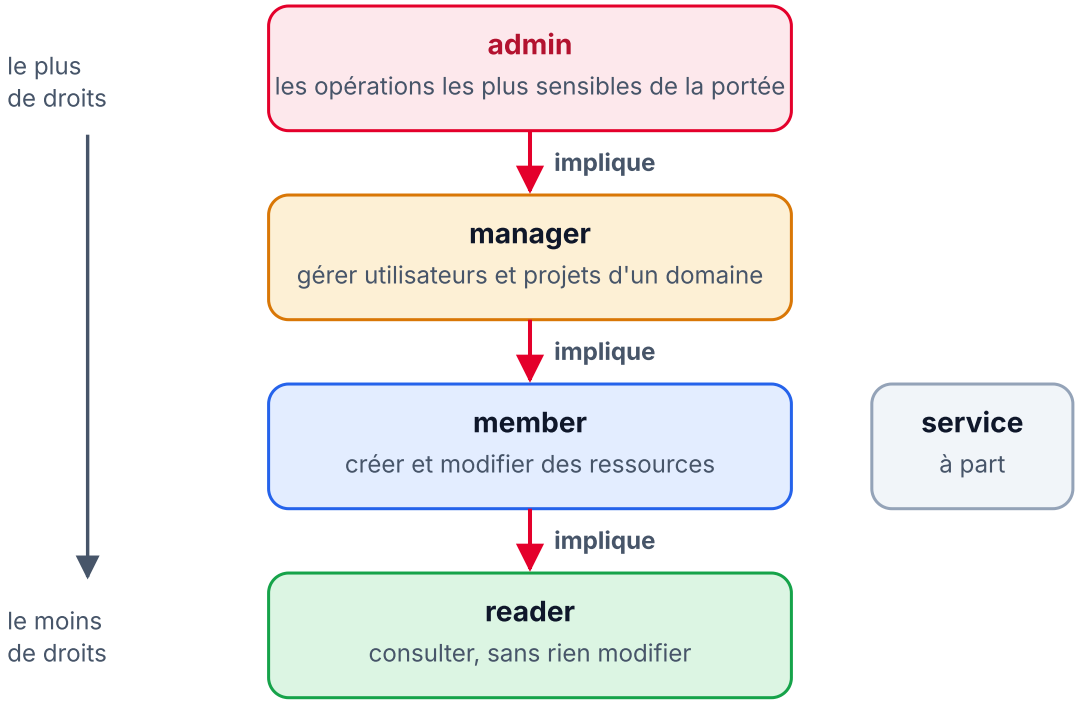
```
openstack user create --domain nord \
  --password-prompt --email alice@exemple.fr alice
openstack group create --domain nord developpeurs
openstack group add user \
  --group-domain nord --user-domain nord \
  developpeurs alice
openstack user set --disable alice    # coupe l'accès
```

Type de compte	Particularité
Humain	mot de passe ou connexion fédérée ; à rattacher à des groupes
Service du cloud	un compte par service (nova, neutron…), créé par le charm, rôle `service` ou `admin`
Automatisation	plutôt un **identifiant d'application** qu'un utilisateur dédié

Désactiver plutôt que supprimer : la suppression d'un utilisateur efface aussi ses attributions et ses identifiants d'application ; la désactivation est réversible.

 Approfondi au module 12

Rôles et rôles impliqués



Détenir « admin » donne automatiquement manager, member et reader.
« service » est réservé aux échanges entre services et n'entre pas dans la chaîne.

- **Un rôle est une étiquette.** Keystone ne sait pas ce que « member » permet dans Nova : ce sont les règles de politique de Nova qui lui donnent un sens.
- **Cinq rôles fournis d'office :** `admin`, `member`, `reader` depuis longtemps ; `manager` et `service` depuis la version 2023.2.
- **Rôle impliqué :** une règle « tel rôle entraîne tel autre », résolue par Keystone à l'émission du jeton. Le jeton d'un administrateur contient donc aussi les rôles inférieurs.
- **Rôles supplémentaires :** certains services en définissent — par exemple pour l'équilibrage de charge ou l'orchestration.
- **Rôle propre à un domaine :** un alias local qui implique des rôles globaux ; rarement nécessaire.

```
openstack role list
openstack implied role list
openstack role create auditeur
openstack implied role create auditeur \
  --implied-role reader
```

Créer un rôle ne donne **aucun** droit tant qu'aucune règle de politique ne le mentionne : préférer les rôles par défaut, compris par tous les services.

Affectations de rôles : acteur, rôle, cible

Acteur	Rôle	Cible	Effet
utilisateur alice	**member**	projet « web »	alice crée et gère des ressources dans web
groupe developpeurs	**reader**	projet « production »	tous les développeurs consultent production, sans rien modifier
groupe exploitants	**member**	projet « production », **hérité**	le rôle vaut aussi pour tous les sous-projets, présents et futurs
utilisateur bob	**manager**	domaine « nord »	bob gère utilisateurs, groupes et projets du domaine
groupe plateforme	**admin**	système	administration du cloud entier
utilisateur nova	**service**	projet « services »	Nova s'identifie auprès des autres services

```
# Attribuer
openstack role add --user alice --user-domain nord \
  --project web --project-domain nord member
openstack role add --group exploitants \
  --group-domain nord --project production \
  --project-domain nord --inherited member
openstack role add --user bob --user-domain nord \
  --domain nord manager
openstack role add --group plateforme \
  --group-domain nord --system all admin
```

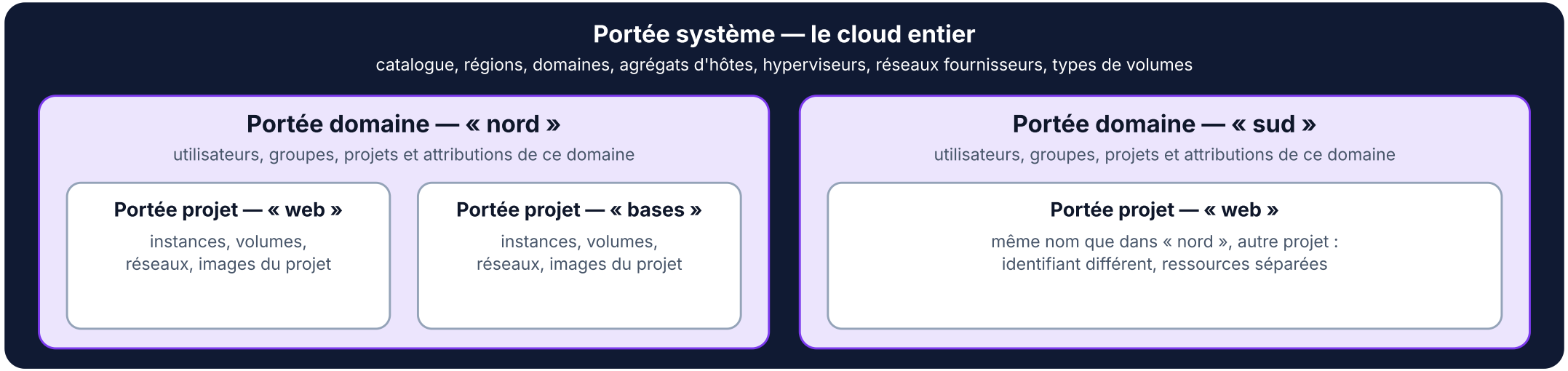
```
# Ce qui a été saisi
openstack role assignment list --names \
  --project web --project-domain nord

# Ce qui s'applique réellement : groupes,
# héritage et rôles impliqués résolus
openstack role assignment list --names \
  --user alice --user-domain nord --effective
```

> **Réflexe de diagnostic :** l'option `--effective` répond à la question « pourquoi cette personne a-t-elle, ou n'a-t-elle pas, ce droit ? ».

 Contenu des jetons : partie 4

Les trois portées



Portée	Demandée avec	Sert à
projet	`--os-project-name`	le travail courant sur les ressources
domaine	`--os-domain-name`	gérer identités et projets d'un domaine
système	`--os-system-scope all`	administrer le cloud
aucune	—	prouver son identité, lister ses projets

- **Un jeton n'a qu'une portée.** Pour agir dans un autre projet, on demande un autre jeton — le client le fait seul, à partir des identifiants.
- **Les droits ne débordent pas :** être `admin` d'un domaine ne donne rien sur le domaine voisin.
- **La portée fait partie de la décision :** une règle de politique peut exiger « rôle reader **et** portée système ».

> Tous les services n'exploitent pas encore la portée système de la même façon : beaucoup d'opérations d'administration passent encore par un jeton de projet avec `admin` (voir partie 6).

 Délégation détaillée : module 9

Les profils par défaut : rôle × portée

Portée	Rôle	Ce que le profil peut faire dans Keystone	Usage type
Système	**admin**	gérer toutes les ressources de Keystone : domaines, catalogue, rôles, tout utilisateur	équipe plateforme
Système	**member, reader**	consulter toutes les ressources ; les deux rôles sont équivalents à ce niveau	audit, supervision, support
Domaine	**admin**	gérer la plupart des aspects de son domaine : projets, utilisateurs, groupes, attributions ; rien au-dehors	responsable d'un locataire
Domaine	**manager**	gérer identités et attributions du domaine, sans pouvoir attribuer ni retirer `admin`, ni modifier le domaine lui-même	délégation sans risque d'escalade
Domaine	**member, reader**	consulter les ressources et informations de son domaine	audit d'un locataire
Projet	**admin**	voir et modifier les données de son projet, poser des étiquettes ; peu de pouvoir dans Keystone	chef de projet technique
Projet	**member, reader**	découvrir les informations de son projet, par exemple ses limites ; rien hors du projet	développeurs, lecture seule

Dans les autres services, la différence entre `member` et `reader` redevient essentielle : un `member` crée une instance, un `reader` ne peut que la lister.

Le rôle `reader` n'est pas anodin : il donne accès en lecture à tout ce que contient la portée. La documentation déconseille de le croire suffisant pour protéger des données sensibles.

 Domaines : module 9 · projets : module 10

Ce que Sunbeam a créé pour vous

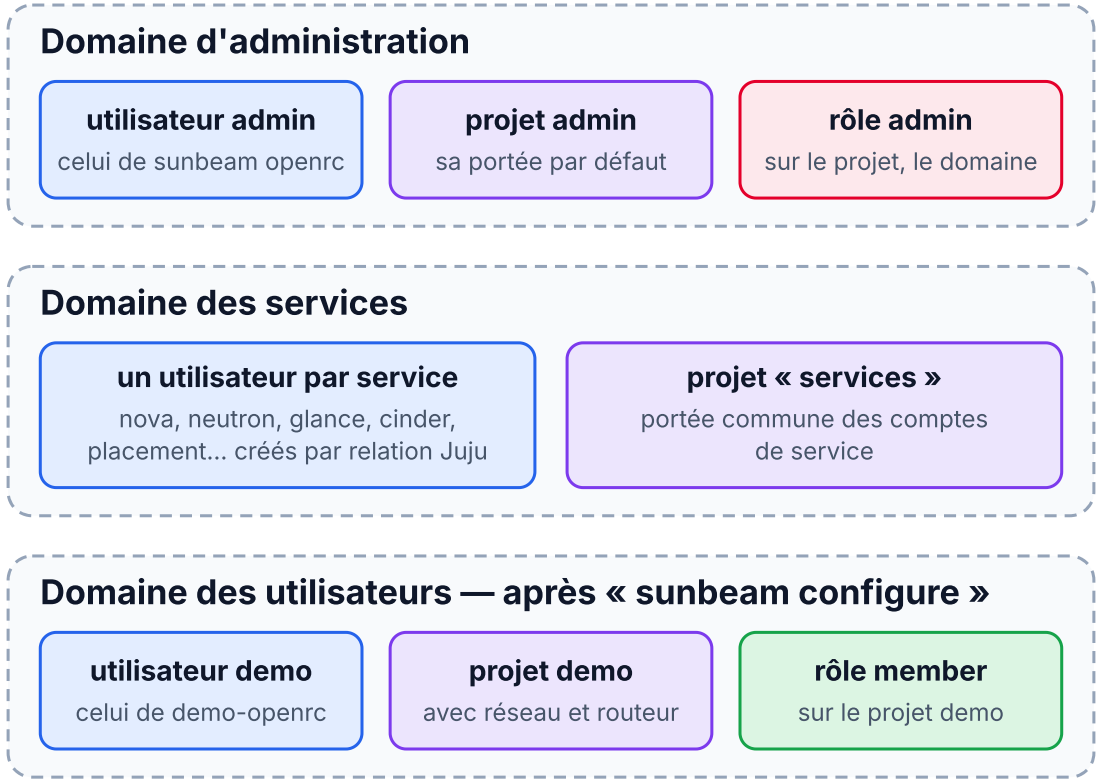


Schéma de principe : noms exacts à relever sur votre cloud.

```
# Relever ce qui existe réellement
openstack domain list
openstack project list --long
openstack user list --long
openstack role list
openstack role assignment list --names
```

- **Séparer humains et services :** les comptes des services vivent dans leur propre domaine, à ne jamais utiliser pour des personnes.
- **Ne pas travailler avec le compte `admin` initial :** créer un domaine pour l'organisation et des comptes nominatifs.
- **Le compte de démonstration** sert à valider l'installation ; il n'a pas vocation à rester sur un cloud de production.

> Le charm expose des actions pour retrouver le mot de passe de l'administrateur ou créer un compte de service : `juju actions keystone` en donne la liste.

 Avec délégation : module 9

Mettre en place un locataire : la séquence complète

```
# 0 · Prérequis : jeton d'administrateur chargé
source admin-openrc

# 1 · Le domaine
openstack domain create --description "Filiale Nord" nord

# 2 · Un projet
openstack project create --domain nord web

# 3 · Des groupes, plutôt que des droits individuels
openstack group create --domain nord web-dev
openstack group create --domain nord web-lecture

# 4 · Les rôles, sur les groupes
openstack role add --group web-dev --group-domain nord \
  --project web --project-domain nord member
openstack role add --group web-lecture --group-domain nord \
  --project web --project-domain nord reader

# 5 · Les personnes, dans les groupes
openstack user create --domain nord --password-prompt alice
openstack group add user --group-domain nord \
  --user-domain nord web-dev alice
```

```
# 6 · Déléguer l'administration du domaine
openstack user create --domain nord --password-prompt bob
openstack role add --user bob --user-domain nord \
  --domain nord manager

# 7 · Vérifier, du point de vue d'alice
openstack --os-username alice --os-user-domain-name nord \
  --os-project-name web --os-project-domain-name nord \
  token issue          # le mot de passe est demandé
```

Étape	Pourquoi dans cet ordre
Domaine d'abord	tout le reste lui appartient
Groupes avant personnes	les droits survivent aux arrivées et aux départs
Délégation	bob gère ensuite son domaine sans solliciter la plateforme
Vérification	un jeton obtenu prouve identité, portée et rôle

> Cette séquence se décrit aussi en Terraform, avec le fournisseur OpenStack : c'est la forme à préférer dès qu'il y a plusieurs locataires.

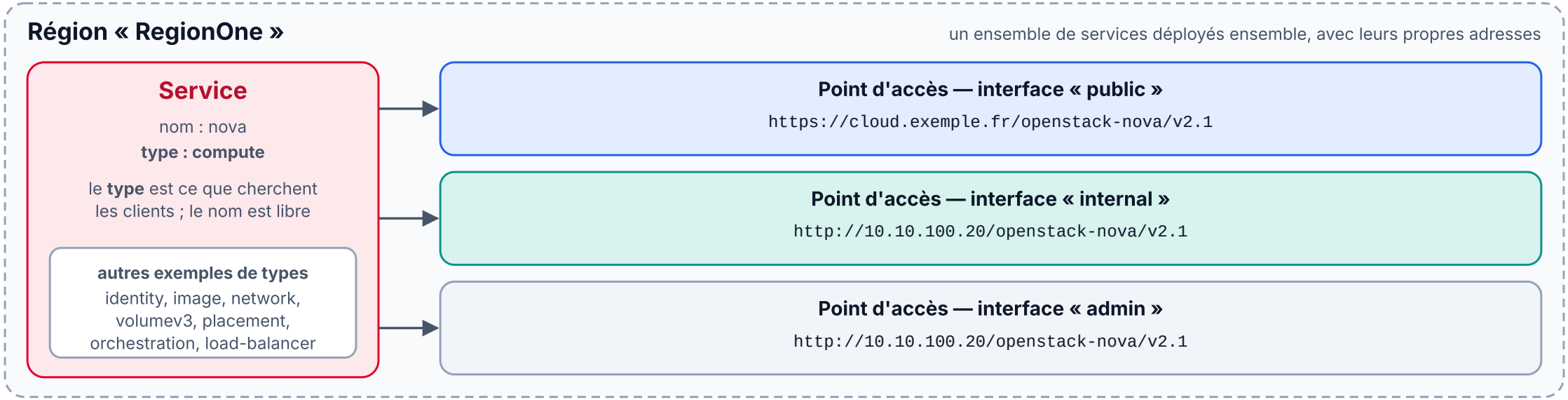
03

Le catalogue de services

L'annuaire du cloud : quels services existent, à quelles adresses, dans quelle région. Aucun client et aucun service ne connaît une adresse « en dur » — tous la lisent ici.

- Services, points d'accès, régions, interfaces
- Le catalogue sur Sunbeam
- Du catalogue à la requête
- Commandes et pannes typiques

Services, points d'accès, régions, interfaces



Interface	Qui l'utilise	Sur quel réseau	Chiffrement
public	**utilisateurs, outils externes, Horizon côté navigateur**	réseau public du cloud, adresse ou nom publié	TLS attendu
internal	**les services entre eux**	réseau interne, non exposé	selon la configuration
admin	**historique ; rarement distingué aujourd'hui**	souvent identique à l'interface interne	selon la configuration

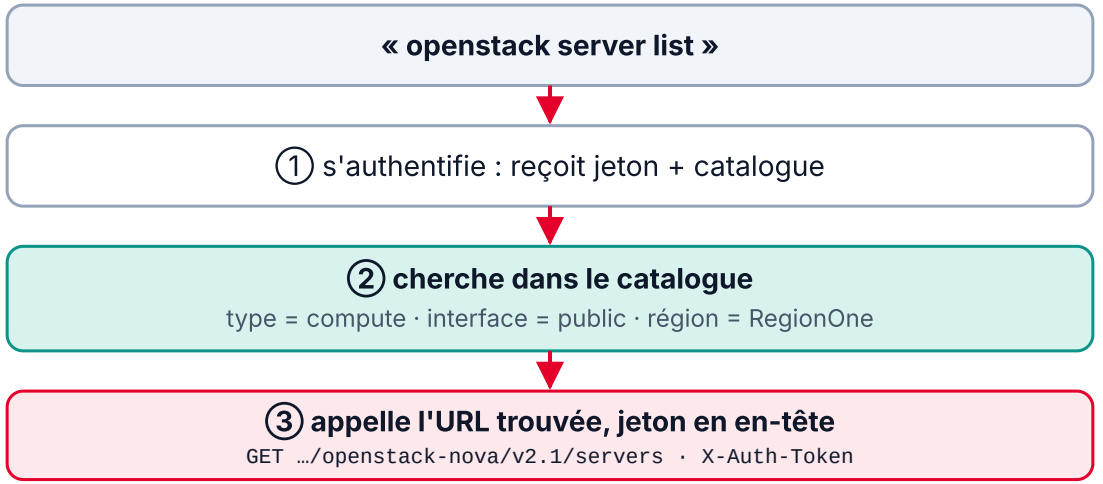
Adresses d'exemple. Un point d'accès est l'association d'un service, d'une région, d'une interface et d'une URL ; il peut être désactivé sans être supprimé.

Du catalogue à la requête

```
# Extrait de la réponse à une demande de jeton
"catalog": [
  {
    "type": "compute",
    "name": "nova",
    "endpoints": [
      { "interface": "public",
        "region": "RegionOne",
        "url": "https://cloud.exemple.fr/openstack-nova/v2.1" },
      { "interface": "internal",
        "region": "RegionOne",
        "url": "http://10.10.100.20/openstack-nova/v2.1" }
    ]
  },
  {
    "type": "image",
    "name": "glance",
    "endpoints": [ … ]
  }
]
```

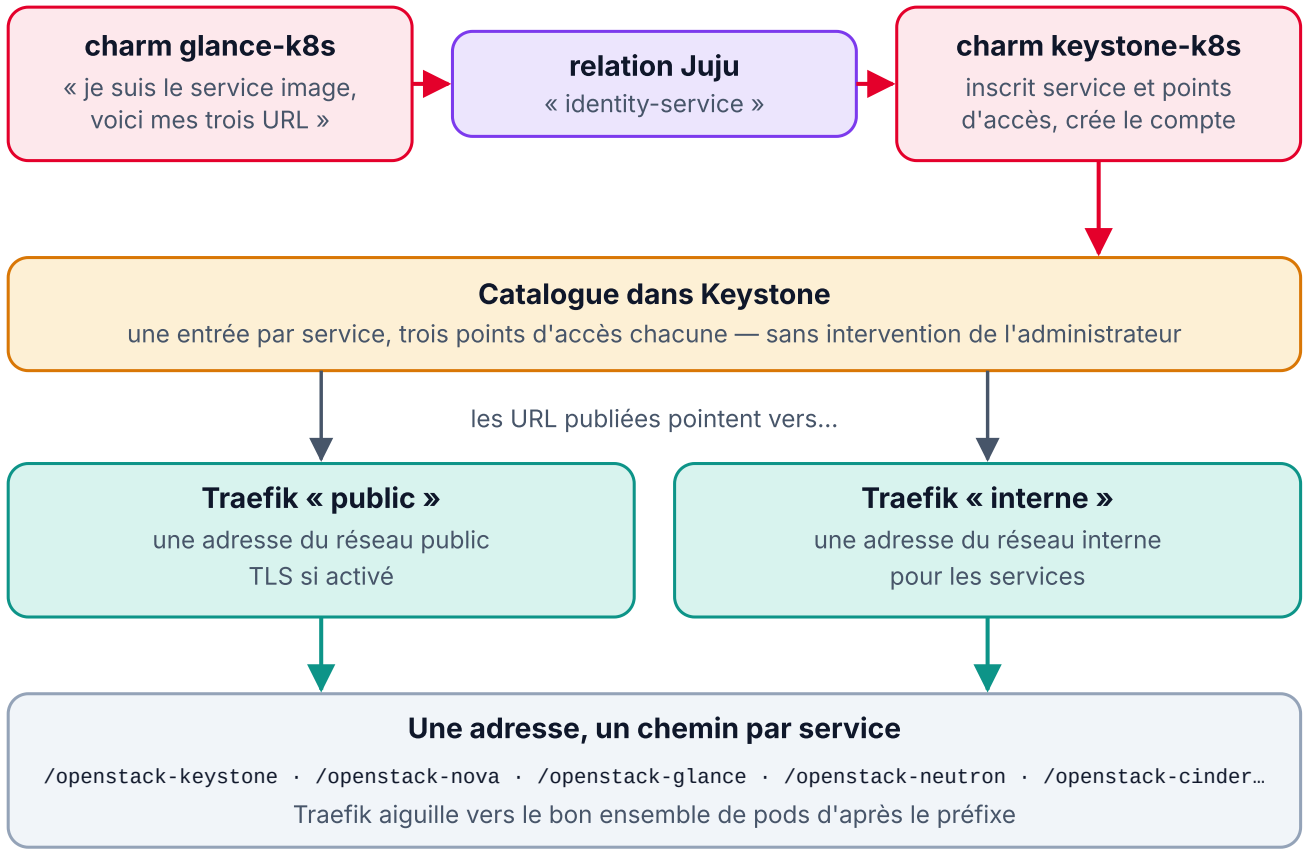
> Le catalogue voyage **dans la réponse d'authentification**, pas dans le jeton lui-même : le jeton reste court, quel que soit le nombre de services.

Ce que fait le client, à chaque commande



- **Trois critères de sélection :** le type de service, l'interface, la région. Les deux derniers se règlent par `OS_INTERFACE` et `OS_REGION_NAME`.
- **Les services font de même :** Nova trouve Neutron, Glance et Cinder dans le catalogue, en demandant l'interface **interne**.
- **Une seule adresse à connaître** pour utiliser tout le cloud : celle de Keystone.

Le catalogue sur Sunbeam



- **Le catalogue se remplit seul :** chaque charm de service déclare ses points d'accès à Keystone par une relation ; l'ajout d'une fonctionnalité Sunbeam ajoute ses entrées.
- **Adresses de service :** prises dans les plages réservées aux API, l'une sur le réseau public, l'autre sur le réseau interne.
- **Un seul port, des chemins :** tous les services partagent l'adresse de Traefik et se distinguent par leur préfixe.
- **TLS :** en activant la fonctionnalité correspondante, les URL publiques passent en `https` et le catalogue est mis à jour par les charms.
- **Nom d'hôte :** on peut publier un nom plutôt qu'une adresse IP ; le manifeste prévoit ce réglage pour chaque entrée.

Catalogue : il se règle par la configuration de Sunbeam ; un `openstack endpoint set` serait réécrit par le charm.

Le nom exact de la relation et des préfixes se lit dans `juju status --relations` et `openstack endpoint list`.

Commandes et pannes typiques

```
# Le catalogue tel que je le reçois
openstack catalog list
openstack catalog show compute

# Les objets sous-jacents (administrateur)
openstack service list
openstack endpoint list --service compute
openstack region list

# Forcer une interface ou une région
openstack --os-interface internal server list
openstack --os-region-name RegionOne image list

# Voir quelle URL est réellement appelée
openstack --debug server list 2>&1 | grep -i "REQ: curl"

# Tester un point d'accès sans client OpenStack
curl -s https://cloud.exemple.fr/openstack-keystone/v3 | jq .
```

Symptôme	Cause probable
« public endpoint for compute service not found »	service absent du catalogue, mauvaise région, ou interface inexistante
délai dépassé depuis un poste, alors que tout marche sur le cloud	le client utilise l'interface interne, ou l'adresse publique n'est pas routée
erreur de certificat	autorité inconnue du client ; nom du certificat différent de l'URL du catalogue
un service n'en trouve pas un autre	point d'accès interne injoignable depuis le pod ; résolution de noms
URL en `http` après activation de TLS	catalogue pas encore mis à jour : vérifier l'état des charms
404 sur une URL correcte	préfixe de chemin oublié ou version d'API absente de l'URL

> **Méthode :** lire l'URL dans le catalogue, l'appeler avec `curl` depuis l'endroit où l'erreur se produit. On sépare ainsi un problème de catalogue d'un problème de réseau.

04

Les jetons d'authentification

La pièce centrale : une preuve datée et scellée de « qui, sur quoi, avec quels rôles », que l'on présente à chaque service au lieu de son mot de passe.

- Jetons et portées
- Anatomie d'un jeton Fernet
- Dimensionner la rotation
- Opérations longues et jetons de service
- Fernet ou JWS
- Obtenir un jeton
- Les clés et leur rotation
- La validation par les services
- Durée de vie et révocation
- Autres façons de s'authentifier

Jetons et portées

Type de jeton	Contient	Sert à	S'obtient en demandant
Sans portée	**l'identité seule : ni rôles, ni catalogue**	prouver qui l'on est ; lister ses projets ; obtenir ensuite des jetons à portée sans redonner son mot de passe	aucune portée
Portée projet	**identité, projet, rôles sur ce projet, catalogue**	le travail courant : instances, volumes, réseaux	un projet et son domaine
Portée domaine	**identité, domaine, rôles sur ce domaine, catalogue**	gérer utilisateurs, groupes et projets du domaine	un domaine
Portée système	**identité, rôles sur le système, catalogue**	opérations sur le cloud entier	le système (« all »)
Issu d'une délégation	**seulement ce que la délégation transmet**	agir au nom d'un autre utilisateur, sur un projet donné	une délégation existante

Un jeton est un « porteur »

Quiconque le détient peut s'en servir jusqu'à son expiration. Il se protège comme un mot de passe : jamais dans un journal, un ticket ou un dépôt.

Un jeton ne se modifie pas

Si les rôles changent, le jeton déjà émis garde l'ancienne liste jusqu'à ce que le service le revalide ; un retrait de rôle provoque sa révocation.

Un jeton est court

Une heure par défaut dans Keystone. Les clients en redemandent un automatiquement à partir des identifiants.

Changer de projet = changer de jeton. C'est ce que fait Horizon quand on sélectionne un autre projet dans son menu : il échange son jeton contre un jeton de l'autre portée.

Obtenir un jeton : la requête et la réponse

```
# POST /v3/auth/tokens
{ "auth": {
    "identity": {
      "methods": ["password"],
      "password": {
        "user": {
          "name": "alice",
          "domain": { "name": "nord" },
          "password": "••••••••"
        }
      }
    },
    "scope": {
      "project": {
        "name": "web",
        "domain": { "name": "nord" }
      }
    }
} }
```

Bloc	Question posée
`identity`	qui es-tu, et comment le prouves-tu ?
`scope`	sur quoi veux-tu agir ?

```
# Réponse : 201 Created
X-Subject-Token: gAAAAABl…          ← le jeton lui-même

{ "token": {
    "methods": ["password"],
    "user": { "id": "3f1c…", "name": "alice",
              "domain": { "name": "nord" } },
    "project": { "id": "9a7e…", "name": "web",
                 "domain": { "name": "nord" } },
    "roles": [ { "name": "member" }, { "name": "reader" } ],
    "issued_at":  "2026-10-09T12:00:00Z",
    "expires_at": "2026-10-09T13:00:00Z",
    "audit_ids": ["Xc1…"],
    "catalog": [ … ]
} }
```

- **Le jeton est dans un en-tête**, pas dans le corps. Le corps décrit ce qu'il certifie.
- `roles` contient déjà les rôles impliqués : `member` entraîne `reader`.
- `methods` garde la trace de la façon dont l'identité a été prouvée.
- `audit_ids` permet de relier un jeton à celui dont il est issu.

> Avec le client : `openstack token issue` fait cette requête ; `--debug` la montre en entier.

Anatomie d'un jeton Fernet

① Le contenu utile, sérialisé de façon compacte (MessagePack)

version	**utilisateur**	**méthodes**	**portée**	**expiration**	**audit**	**selon le cas**
type de contenu	identifiant	password, token…	projet, domaine ou système	date et heure	identifiants de suivi	délégation, fédération

chiffré avec la clé primaire

② Le jeton Fernet

version	**horodatage**	**vecteur d'initialisation**	**contenu chiffré**	**signature**
1 octet	date de création	aléatoire	AES en mode CBC	HMAC SHA-256 de tout ce qui précède

encodé en texte

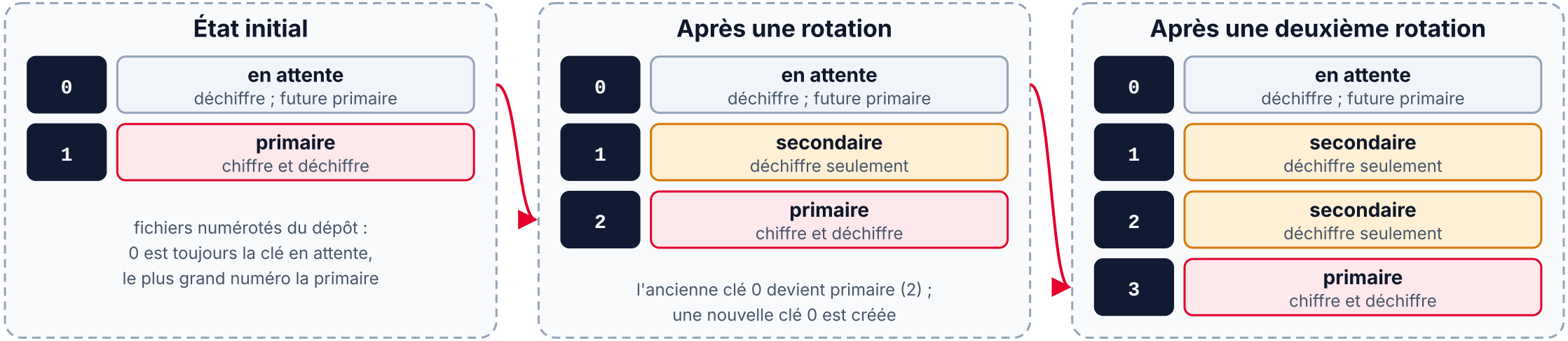
```
gAAAAABnBlV8kq3X0y1mQJ4R7fK2hT9wZc5uE… — moins de 250 octets, quel que soit le nombre de services ou de rôles
```

Ce que le jeton ne contient **pas** : les rôles et le catalogue. Ils sont recalculés par Keystone à chaque validation, ce qui explique qu'un jeton Fernet ne puisse être vérifié que par Keystone.

Rien n'est stocké : Keystone ne garde aucune trace des jetons émis. Il lui suffit de la clé pour déchiffrer et vérifier celui qu'on lui présente — d'où l'absence de table de jetons à purger.

Chiffré, donc opaque : impossible de lire ou de modifier un jeton sans les clés. Qui détient les clés, en revanche, peut **fabriquer** des jetons pour n'importe qui.

Les clés Fernet et leur rotation



Clé	Numéro	Chiffre ?	Déchiffre ?	Rôle
primaire	le plus élevé	oui	oui	émet tous les nouveaux jetons
secondaire	entre les deux	non	oui	ancienne primaire : valide les jetons encore en circulation
en attente	0	non	oui	future primaire, déjà distribuée partout

- **Pourquoi une clé en attente :** quand elle devient primaire, tous les serveurs la connaissent déjà et savent valider les jetons qu'elle émet.
- **Dépôt de clés :** un répertoire de fichiers, hors base de données, qui doit être **identique** sur tous les serveurs Keystone.

> Rotation et distribution vont ensemble : une rotation non distribuée, et une partie des serveurs refuse des jetons valides.

Dimensionner la rotation — et ce qu'en fait Sunbeam

La règle de calcul

nombre de clés actives =

`(durée de vie du jeton ÷ fréquence de rotation) + 2`

Les deux clés supplémentaires : celle en attente, et une de marge.
Si des jetons expirés restent acceptés pour les opérations longues, ajouter cette fenêtre à la durée de vie dans le calcul.

Durée de vie	Fenêtre	Rotation	Clés actives
1 h	—	toutes les heures	1 + 2 = **3**
24 h	—	toutes les 6 h	4 + 2 = **6**
1 h	48 h	toutes les 12 h	5 + 2 = **7**

Trop peu de clés : une clé encore nécessaire est supprimée, et des jetons non expirés deviennent invérifiables — erreurs 401 sans raison apparente.

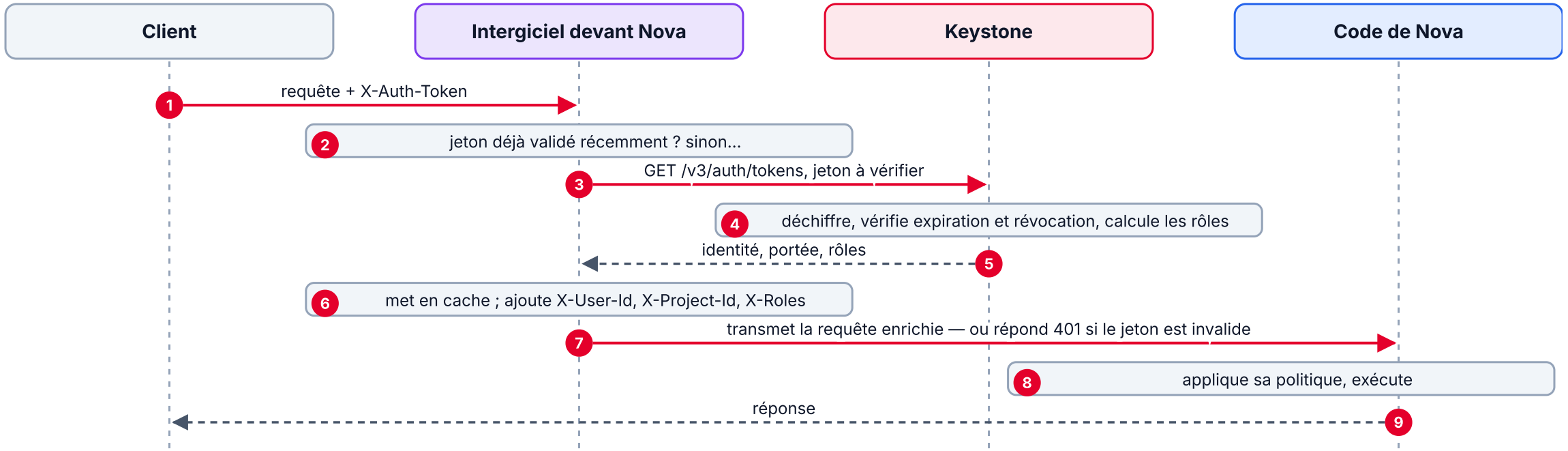
Option de keystone.conf	Rôle
`[token] provider`	format des jetons : `fernet` par défaut
`[token] expiration`	durée de vie, en secondes ; 3600 par défaut en amont
`[token] allow_expired_window`	durée pendant laquelle un jeton expiré reste récupérable pour un service
`[fernet_tokens] key_repository`	répertoire des clés
`[fernet_tokens] max_active_keys`	nombre de clés conservées

Sur Sunbeam

- **Vous ne lancez pas** `keystone-manage fernet_rotate` : le charm crée le dépôt, fait tourner les clés et les distribue à tous les pods.
- **Les clés sont partagées par Juju**, pas par un volume commun : un nouveau pod les reçoit avant de servir.
- **Les valeurs effectives** se lisent dans le pod ; les réglages exposés, dans `juju config keystone`.

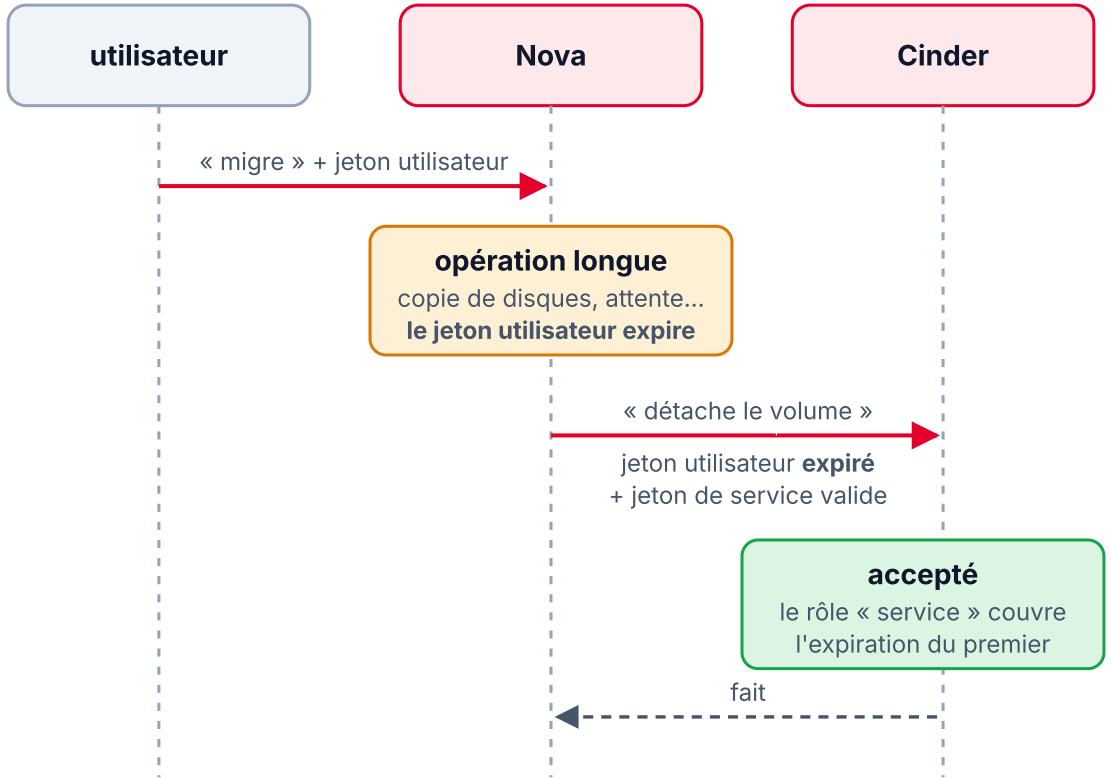
```
sudo k8s kubectl -n openstack exec keystone-0 \
  -c keystone -- grep -A4 '^\[token\]' \
  /etc/keystone/keystone.conf
```

La validation par les services



- **Chaque service embarque le même composant** — `keystonemiddleware` — placé devant son API. C'est lui qui parle à Keystone ; le code du service ne voit que des en-têtes.
- **Le service a son propre compte** dans Keystone : il s'authentifie pour avoir le droit de valider les jetons des autres.
- **Cache :** le résultat d'une validation est gardé quelques minutes. C'est aussi le délai pendant lequel un jeton révoqué peut encore passer.
- **Section à connaître** dans la configuration de tout service : `[keystone_authtoken]` — adresse de Keystone, compte de service, cache.

Opérations longues et jetons de service



Le problème

- Un jeton dure une heure ; une migration à chaud ou la création d'une grosse image peut durer plus.
- En cours de route, Nova doit encore appeler Cinder, Neutron ou Glance **au nom de l'utilisateur**.

La solution : deux jetons dans la requête

- **Le jeton de l'utilisateur**, dans `X-Auth-Token` : il dit pour qui l'on agit.
- **Le jeton du service appelant**, dans `X-Service-Token` : il prouve que la requête vient bien d'un service du cloud.
- Si le second est valide et porte le rôle `service`, le service appelé accepte le premier **même expiré**, dans la limite de `allow_expired_window`.

> **C'est pour cela que le rôle `service` existe**, à part de la chaîne des autres : il ne donne aucun droit sur les ressources, seulement cette confiance entre services.

Symptôme d'une mauvaise configuration : des opérations longues qui échouent en 401 au bout d'une heure pile.

Durée de vie et révocation

Ce qui invalide un jeton avant son expiration

Événement	Jetons touchés
révocation explicite	**ce jeton, et ceux qui en sont issus**
changement de mot de passe	**tous ceux de l'utilisateur**
utilisateur désactivé ou supprimé	**tous les siens**
rôle retiré	**ceux qui portaient ce rôle sur cette cible**
projet ou domaine désactivé	**tous ceux de cette portée**
suppression des clés Fernet	**tous les jetons du cloud**

```
openstack token revoke gAAAAABn…
```

> **Mécanisme :** Keystone n'ayant pas de liste des jetons, il enregistre des **événements de révocation** — « tout ce qui concerne cet utilisateur avant telle heure » — consultés à chaque validation.

Régler la durée de vie

Durée	Avantage	Inconvénient
courte : 1 h	un jeton volé sert peu de temps	plus d'authentifications ; dépend des jetons de service
longue : 12 à 24 h	confort des sessions Horizon et des scripts	fenêtre d'abus plus grande ; plus de clés à garder

Bonnes pratiques

- **Garder la durée courte** ; pour l'automatisation, voir les identifiants d'application, slide suivante.
- **Ne jamais copier un jeton** dans un script : y mettre de quoi en obtenir un.
- **Délai de prise d'effet :** une révocation n'est vue par un service qu'à l'expiration de son cache de validation.
- **Fuite des clés suspectée :** remplacer tout le dépôt de clés — cela déconnecte tout le monde, c'est le but.

Fernet ou JWS : deux fournisseurs de jetons

	Fernet — le défaut	JWS
Apparu en	**Kilo**	Stein
Principe	**contenu chiffré et signé avec une clé symétrique**	contenu **signé**, non chiffré, avec une paire de clés asymétriques
Algorithmes	**AES pour le chiffrement, HMAC SHA-256 pour l'intégrité**	ES256 uniquement
Lisible par un tiers	**non : opaque**	oui : le contenu se décode sans clé
Stockage côté serveur	**aucun**	aucun
Clés à protéger	**le dépôt entier : qui le lit peut émettre des jetons**	la clé privée ; les clés publiques peuvent circuler
Validation	**toujours auprès de Keystone**	recommandée auprès de Keystone : le format du contenu n'est pas un contrat
Rotation	**primaire, secondaires, en attente**	ajout et retrait de paires de clés

Dans la pratique : Fernet est le choix de presque tous les déploiements, et celui de Sunbeam. JWS intéresse les architectures où distribuer une clé secrète à tous les sites pose problème.

Pour mémoire : les anciens formats — jetons UUID stockés en base, jetons PKI de plusieurs kilo-octets — ont disparu. Leurs défauts expliquent la conception de Fernet : rien à stocker, taille fixe.

 Identifiants d'application : module 12

Autres façons de s'authentifier

Méthode	Principe	Usage	À savoir
Mot de passe	**nom, domaine, mot de passe**	humains, base locale ou LDAP	à ne jamais mettre dans un script
Jeton	**échanger un jeton contre un autre, d'une autre portée**	changement de projet	le nouveau n'expire pas plus tard que l'ancien
Identifiant d'application	**un secret créé par l'utilisateur, limité à un projet et à un sous-ensemble de ses rôles**	scripts, CI, Terraform, applications	révocable sans toucher au mot de passe ; meurt avec l'utilisateur
Règles d'accès	**restreint un identifiant d'application à certains appels d'API**	moindre privilège	liste de couples méthode + chemin
Délégation	**un utilisateur confie certains de ses rôles sur un projet à un autre**	Heat agissant plus tard au nom de l'utilisateur	avec ou sans usurpation d'identité
Second facteur	**code à usage unique (TOTP) exigé en plus du mot de passe**	comptes sensibles	règles définies par utilisateur
Externe, fédérée	**le serveur web ou un fournisseur d'identité atteste l'identité**	connexion unique d'entreprise	voir la partie 5

```
# Créer un identifiant d'application, en lecture seule, 90 jours
openstack application credential create ci-lecture \
  --role reader --expiration 2027-01-07T00:00:00
# → noter « id » et « secret » : le secret n'est montré qu'une fois
```

```
# S'en servir
export OS_AUTH_TYPE=v3applicationcredential
export OS_APPLICATION_CREDENTIAL_ID=<id>
export OS_APPLICATION_CREDENTIAL_SECRET=<secret>
openstack server list
```

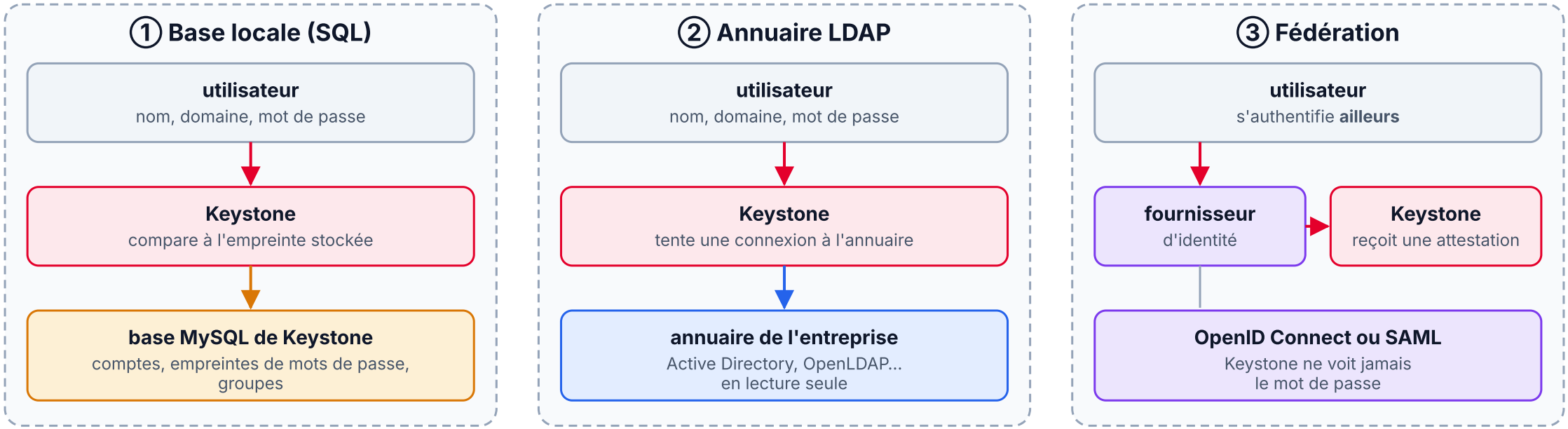
05

Les sources d'identité

D'où viennent les utilisateurs ? De la base de Keystone, de l'annuaire de l'entreprise, ou d'un fournisseur d'identité qui les authentifie à sa place.

- Trois familles de sources
- LDAP : un annuaire par domaine
- La fédération : principe
- La fédération sur Sunbeam
- Comparatif
- La base locale et sa politique de sécurité
- LDAP sur Sunbeam
- Une connexion fédérée pas à pas
- Les règles de correspondance

Trois familles de sources



Point commun : seule l'**identité** change de source. Projets, rôles et attributions restent toujours dans la base de Keystone — l'annuaire ne sait rien d'OpenStack.

Elles se combinent, par domaine : un domaine local pour les services et le secours, un domaine LDAP pour les employés, un domaine fédéré pour les partenaires.

5 · LES SOURCES D'IDENTITÉ Politique ANSSI : module 11

La base locale et sa politique de sécurité

Comment sont gardés les mots de passe

- Jamais en clair : une **empreinte** calculée par un algorithme volontairement lent — bcrypt par défaut — avec un sel propre à chaque compte.
- L'historique des empreintes est conservé, pour interdire la réutilisation.

Ce que la base locale sait faire

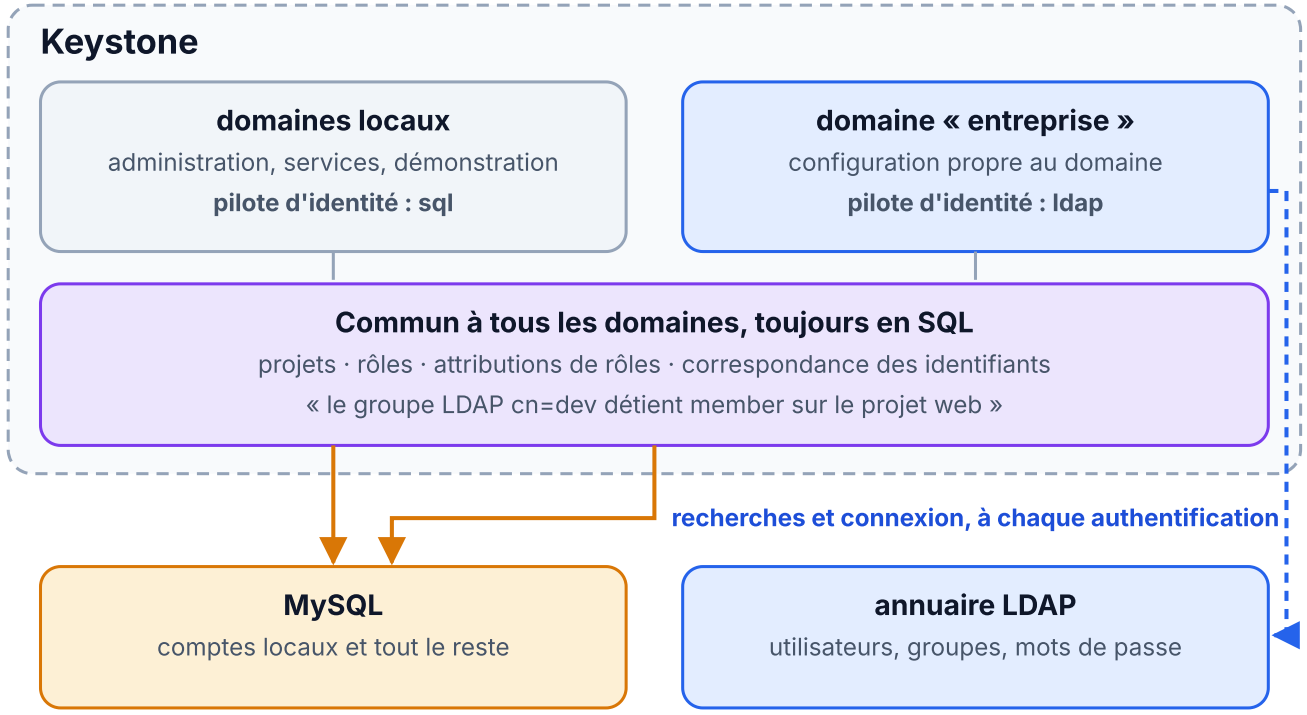
- Créer, modifier, désactiver des comptes et des groupes par l'API.
- Laisser l'utilisateur changer lui-même son mot de passe.
- Appliquer une politique de sécurité : verrouillage, expiration, complexité.
- Exiger un second facteur, compte par compte.

Ces règles ne valent que pour la base locale. Pour LDAP ou la fédération, c'est la source externe qui applique sa propre politique.

Option de `[security_compliance]`	Effet
`lockout_failure_attempts`	nombre d'échecs avant verrouillage
`lockout_duration`	durée du verrouillage
`password_expires_days`	durée de validité d'un mot de passe
`unique_last_password_count`	nombre d'anciens mots de passe interdits
`minimum_password_age`	délai minimal entre deux changements
`password_regex`	règle de complexité
`disable_user_account_days_inactive`	désactivation après inactivité
`change_password_upon_first_use`	changement imposé à la première connexion

- **Exempter un compte :** des options par utilisateur lèvent telle ou telle règle — indispensable pour les comptes de service, qui ne doivent ni expirer ni se verrouiller.
- **Sur Sunbeam :** ces options ne sont réglables que si le charm les expose ; à vérifier dans `juju config keystone`.

LDAP : un annuaire par domaine



Keystone lit l'annuaire à chaque authentification et à chaque liste d'utilisateurs : il n'en fait pas de copie, et n'y écrit rien.

- **Une configuration par domaine :** chaque domaine peut pointer vers un annuaire différent, ou vers une branche différente du même.
- **Authentification :** Keystone cherche l'entrée de l'utilisateur, puis tente de s'y connecter avec le mot de passe fourni. C'est l'annuaire qui dit oui ou non.
- **Lecture seule :** on ne crée ni ne modifie de compte LDAP par l'API d'OpenStack.
- **Identifiants :** Keystone attribue à chaque entrée LDAP un identifiant stable, gardé dans une table de correspondance.
- **Groupes :** ceux de l'annuaire sont visibles ; on leur attribue des rôles comme à des groupes locaux.

Réglage LDAP	Rôle
`url`, `user`, `password`	serveur et compte de recherche
`suffix`	racine de l'annuaire
`user_tree_dn`, `user_filter`	où et quels utilisateurs
`user_name_attribute`	attribut servant de nom de connexion
`group_tree_dn`	où chercher les groupes

 Un annuaire par domaine : module 9

LDAP sur Sunbeam

Prérequis : l'annuaire joignable depuis les nœuds de contrôle ; un compte de recherche en lecture ; le certificat de l'autorité si la liaison est en TLS.

```
# entreprise.yaml — les quatre clés obligatoires
url: ldaps://ldap.exemple.fr:636
user: cn=keystone,ou=services,dc=exemple,dc=fr
password: "••••••••"
suffix: dc=exemple,dc=fr
# puis, selon l'annuaire :
user_tree_dn: ou=personnes,dc=exemple,dc=fr
user_objectclass: inetOrgPerson
user_id_attribute: uid
user_name_attribute: uid
group_tree_dn: ou=groupes,dc=exemple,dc=fr
group_objectclass: groupOfNames
```

Le compte de recherche n'a besoin que de lire utilisateurs et groupes. Son mot de passe est dans ce fichier : le traiter comme un secret et le supprimer après usage.

```
# 1 · Activer la fonctionnalité, une fois
sunbeam enable ldap

# 2 · Raccorder un domaine à l'annuaire
sunbeam ldap add-domain \
  --domain-config-file ./entreprise.yaml \
  --ca-cert-file ./ca-entreprise.pem entreprise

# 3 · Vérifier
sunbeam ldap list-domains
openstack user list --domain entreprise
openstack group list --domain entreprise

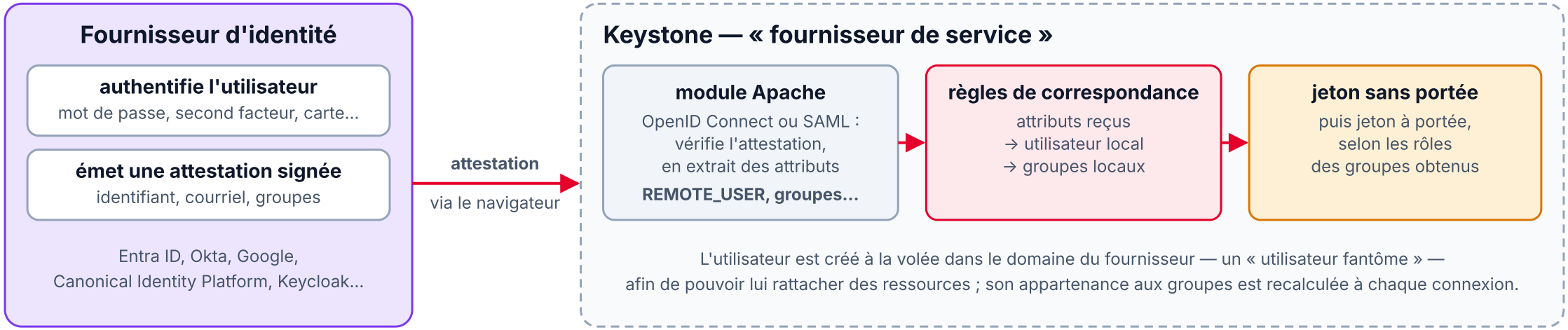
# 4 · Donner des droits — aux groupes de l'annuaire
openstack role add --group dev --group-domain entreprise \
  --project web --project-domain nord member

# Modifier, retirer
sunbeam ldap update-domain \
  --domain-config-file ./entreprise.yaml entreprise
sunbeam ldap remove-domain entreprise
```

Retirer ne supprime que la liaison avec l'annuaire : le domaine, ses projets et ses attributions restent. Pour tout enlever, `openstack domain delete .`

 Approfondi au module 11

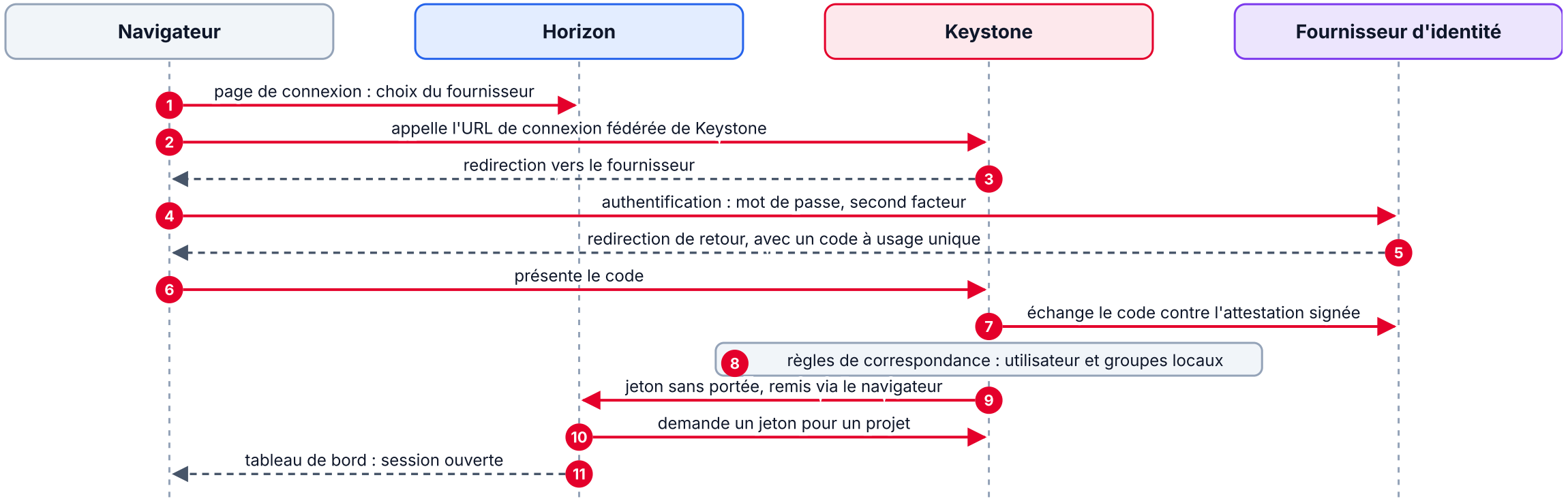
La fédération : principe



Objet Keystone	Ce qu'il représente	Exemple
Fournisseur d'identité	**la source externe, reconnue par son identifiant distant ; rattaché à un domaine**	« entra-groupe », domaine « partenaires »
Protocole	**la façon de dialoguer avec lui**	`openid` ou `saml2`
Correspondance	**les règles qui traduisent les attributs reçus en utilisateur et groupes locaux**	« groupe cloud-dev → groupe developpeurs »

Ce que l'on y gagne : connexion unique, second facteur et cycle de vie des comptes gérés par l'entreprise ; un départ coupe l'accès au cloud sans rien faire dans OpenStack.

Une connexion fédérée, pas à pas



Le mot de passe ne passe jamais par OpenStack : il est saisi chez le fournisseur d'identité, qui ne renvoie qu'une attestation signée, à durée de vie très courte.

Et en ligne de commande ? Le parcours par navigateur ne s'y prête pas : l'usage courant est de se connecter une fois à Horizon, d'y créer un **identifiant d'application**, et de s'en servir pour les scripts.

5 · LES SOURCES D'IDENTITÉ Approfondi au module 11

La fédération sur Sunbeam

Fournisseur	Type Sunbeam	Protocole documenté
Canonical Identity Platform	`canonical`	OpenID Connect
Google	`google`	OpenID Connect
Okta	`okta`	OpenID Connect
Entra ID	`entra`	OpenID Connect
Autres, SAML2	non documentés	`--help` de votre version

Prérequis

- **TLS** sur les points d'accès publics — par la fonctionnalité TLS de Sunbeam (autorité fournie ou Vault) : la plupart des fournisseurs exigent une URL de retour en `https`.
- **Un nom d'hôte** pour le cloud : certains fournisseurs refusent une adresse IP.
- Côté fournisseur : une application déclarée, avec son identifiant client et son secret.

```
# entra.yaml
client-id: 5d1f…
client-secret: "••••••••"
microsoft-tenant: 8a2c…
```

```
# 1 · URL de retour à déclarer chez le fournisseur
sunbeam identity provider get-oidc-redirect-url \
https://cloud.exemple.fr/openstack-keystone/v3/\
OS-FEDERATION/protocols/openid/redirect_uri

# 2 · Déclarer le fournisseur : type, protocole, nom
sunbeam identity provider add \
  entra openid entra-groupe --config entra.yaml
sunbeam identity provider list

# 3 · Côté Keystone : domaine, fournisseur, groupe, rôle
openstack domain create partenaires
REMOTE_ID=$(sunbeam identity provider list --format=yaml \
  | yq -r '."entra-groupe".issuer_url')
openstack identity provider create \
  --remote-id $REMOTE_ID --domain partenaires entra-groupe
openstack group create --domain partenaires federes
openstack role add --group federes \
  --group-domain partenaires --project web \
  --project-domain nord member

# 4 · Règles de correspondance, puis protocole
openstack mapping create --rules regles.json entra_openid
openstack federation protocol create \
  --identity-provider entra-groupe \
  --mapping entra_openid openid
```

> Sunbeam règle Apache et les secrets ; les objets Keystone restent à créer par l'administrateur. Tant que la correspondance manque, le fournisseur apparaît dans Horizon mais personne ne peut se connecter.

Les règles de correspondance

```
# regles.json — règle simple : tout utilisateur authentifié
[
  {
    "local": [
      { "user":  { "name": "{0}" } },
      { "group": { "name": "federes",
                   "domain": { "name": "partenaires" } } }
    ],
    "remote": [
      { "type": "REMOTE_USER" }
    ]
  }
]
```

```
# Règle conditionnelle : seulement un groupe du fournisseur
"remote": [
  { "type": "OIDC-preferred_username" },
  { "type": "OIDC-groups", "any_one_of": ["cloud-dev"] }
]
```

Élément	Signification
`remote`	conditions sur les attributs transmis par le fournisseur ; toutes doivent être satisfaites
`local`	ce que l'on en fait : nom de l'utilisateur local, groupes d'affectation
`{0}`, `{1}`...	valeur du premier, du deuxième attribut de `remote`
`any_one_of`	la valeur doit figurer dans la liste
`not_any_of`	la valeur ne doit pas y figurer

- **Les règles sont évaluées dans l'ordre ;** un utilisateur peut cumuler les groupes de plusieurs règles satisfaites.
- **Aucune règle satisfaite :** connexion refusée, même si l'authentification chez le fournisseur a réussi.
- **Une seule correspondance** par couple fournisseur + protocole ; une même correspondance peut servir à plusieurs fournisseurs.
- **Noms des attributs :** ils dépendent du module et du fournisseur ; les journaux de Keystone en mode détaillé montrent ceux qui arrivent réellement.

> **Piège classique :** une règle trop large donne un rôle à **tout** compte du fournisseur. Toujours conditionner à un groupe dédié au cloud.

Comparatif des sources d'identité

	Base locale	LDAP	Fédération
Où est le mot de passe	**empreinte dans la base de Keystone**	dans l'annuaire ; il transite par Keystone	chez le fournisseur ; Keystone ne le voit jamais
Création des comptes	**par l'API d'OpenStack**	dans l'annuaire	chez le fournisseur ; utilisateur local créé à la première connexion
Second facteur	**TOTP de Keystone**	non, sauf dispositif de l'annuaire	celui du fournisseur
Connexion unique	**non**	même mot de passe, mais ressaisi	oui
Ligne de commande	**directe**	directe	par identifiant d'application
Départ d'un employé	**à traiter dans OpenStack**	automatique, dès la désactivation dans l'annuaire	automatique pour les nouvelles connexions
Dépendance	**aucune**	l'annuaire doit être joignable à chaque authentification	le fournisseur doit être joignable à la connexion
Sur Sunbeam	**d'origine**	`sunbeam enable ldap`	`sunbeam identity provider add`
Usage type	**comptes de service, secours, maquettes**	employés, quand l'annuaire est la référence	employés et partenaires, avec connexion unique

Toujours garder un compte local de secours, hors annuaire et hors fédération : le jour où la source externe est en panne, c'est le seul moyen d'entrer.

Avec la fédération, penser aux identifiants d'application : ils survivent tant que l'utilisateur local existe ; prévoir leur nettoyage quand un compte est retiré chez le fournisseur.

06

Les politiques d'accès

Keystone dit qui vous êtes et quels rôles vous portez. C'est chaque service, règle par règle, qui décide si cela suffit pour l'opération demandée.

- De la requête à la décision
- Règles par défaut et surcharges
- Qui peut faire quoi dans Keystone
- Bonnes pratiques
- Anatomie d'une règle
- Portées et nouveaux rôles par défaut
- Chaque service a ses politiques

De la requête à la décision



- **La décision est locale :** Nova applique les règles de Nova, Cinder celles de Cinder. Keystone n'est pas consulté pour décider, seulement pour valider le jeton.
- **Une règle par opération d'API :** plusieurs centaines dans un cloud complet.
- **401 ou 403 ?** 401 : le jeton est absent, expiré ou invalide — on ne sait pas qui vous êtes. 403 : on le sait, et la règle dit non.
- **Le rôle seul ne suffit pas :** la plupart des règles vérifient aussi que la cible appartient à la portée du jeton.

 Le piège du rôle admin : module 9

Anatomie d'une règle

```
# Règles par défaut de Keystone — extraits
"admin_required":         "role:admin or is_admin:1"
"service_role":           "role:service"

"identity:list_users":    "(rule:admin_required)
                           or (role:reader and system_scope:all)
                           or (role:reader and domain_id:%(target.domain_id)s)"

"identity:get_user":      "(rule:admin_required) or (role:reader and system_scope:all)
                           or (role:reader and token.domain.id:%(target.user.domain_id)s)
                           or user_id:%(target.user.id)s"

"identity:create_project": "(rule:admin_required)
                            or (role:manager and domain_id:%(target.project.domain_id)s …)"
```

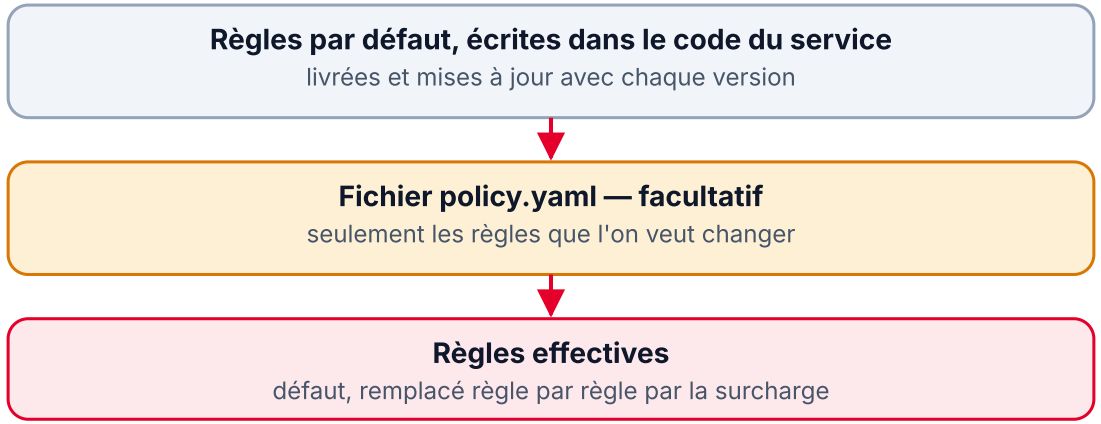
Élément	Signification
`role:reader`	le jeton porte ce rôle
`rule:admin_required`	renvoi à une autre règle, réutilisable
`system_scope:all`	le jeton est de portée système
`domain_id:%(target.domain_id)s`	le domaine du jeton est celui de la cible
`user_id:%(target.user.id)s`	l'appelant est l'utilisateur visé
`and` , `or` , `not` , **parenthèses**	combinaisons logiques

Lecture de `identity:get_user`

- un **administrateur** ; ou
- un **lecteur de portée système** ; ou
- un **lecteur du domaine** de l'utilisateur consulté ; ou
- **l'utilisateur lui-même**, quel que soit son rôle.

> **À remarquer :** `admin_required` ne mentionne aucune portée. Dans Keystone, le rôle `admin` reste très puissant **quelle que soit la cible** sur laquelle il est détenu : ne jamais l'attribuer « sur un projet » en pensant le cantonner.

Règles par défaut et surcharges



- **Pas de fichier, pas de problème :** un service sans `policy.yaml` applique les règles de sa version. C'est l'état recommandé.
- **Format :** YAML ; l'ancien format JSON est déprécié depuis Wallaby.
- **Rechargement :** le fichier est relu à chaud, sans redémarrage.
- **Chaque surcharge est une dette :** à chaque montée de version, il faut vérifier qu'elle a encore un sens face aux nouveaux défauts.

```
# Voir les règles par défaut d'un service
oslopolicy-sample-generator --namespace keystone

# Voir les règles effectives, surcharges comprises
oslopolicy-policy-generator --namespace keystone

# Repérer les surcharges devenues inutiles
oslopolicy-list-redundant --namespace keystone
```

```
# policy.yaml — exemple de surcharge
# autoriser le rôle « auditeur » à lister les utilisateurs
"identity:list_users": "(rule:admin_required)
   or (role:reader and system_scope:all)
   or role:auditeur"
```

Sur Sunbeam

- Les fichiers de configuration sont dans les pods et **réécrits par les charms** : un `policy.yaml` déposé à la main ne survit pas.
- Une surcharge passe par ce que le charm expose ; à défaut, s'en tenir aux rôles et règles par défaut.
- Les outils `oslopolicy-*` s'exécutent dans le conteneur du service, par `kubectl exec`.

Portées et nouveaux rôles par défaut

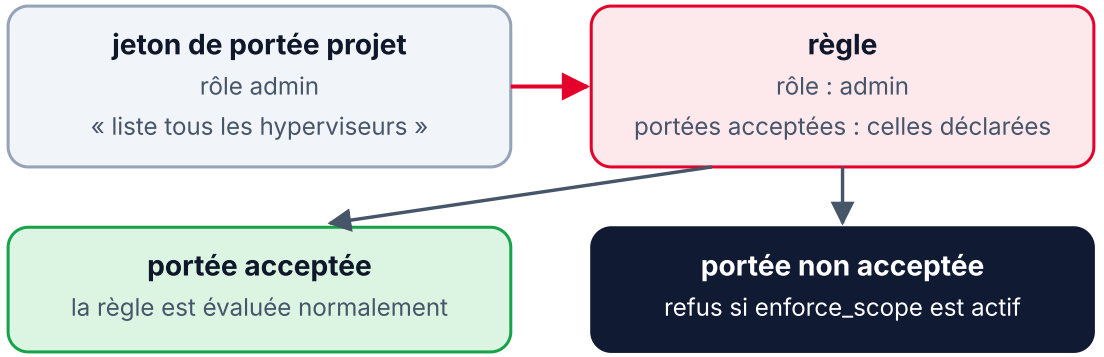
D'où l'on vient

- Longtemps, les règles se résumaient à « `admin` ou propriétaire ». Le rôle `admin`, attribué sur **n'importe quel** projet, valait administrateur du cloud entier.
- Pas de lecture seule : pour voir, il fallait pouvoir modifier.

Ce qui a changé — le « RBAC sécurisé »

- **Des rôles gradués** partout : `reader`, `member`, `manager`, `admin`.
- **Des portées déclarées :** chaque règle annonce les portées de jeton qu'elle accepte (`scope_types`) — système, domaine, projet.
- **Deux interrupteurs**, dans la section `[oslo_policy]` de chaque service :

Option	Quand elle est active
`enforce_new_defaults`	seules les nouvelles règles s'appliquent ; les anciennes, plus permissives, ne sont plus acceptées en parallèle
`enforce_scope`	un jeton de la mauvaise portée est refusé, même avec le bon rôle



- **État des lieux :** ces options sont actives par défaut dans les versions récentes de la bibliothèque, mais chaque service a avancé à son rythme. Vérifier la valeur **service par service**.
- **Conséquence pratique :** selon le service, une opération d'administration attend un jeton de portée système **ou** un jeton de projet avec `admin`.
- **En cas de 403 inattendu** pour un administrateur : regarder d'abord la **portée** du jeton, pas le rôle.

```
sudo k8s kubectl -n openstack exec nova-0 -c nova-api -- \
  grep -A3 '^\[oslo_policy\]' /etc/nova/nova.conf
```

> Noms de pod et de conteneur à adapter : `kubectl get pods -n openstack` les donne.

6 · POLITIQUES D'ACCÈS Gestionnaire en détail : module 9

Qui peut faire quoi dans Keystone

Opération	admin système	lecteur système	admin de domaine	manager de domaine	lecteur de domaine	membre de projet
Créer, supprimer un domaine	oui	—	—	—	—	—
Modifier le catalogue	oui	—	—	—	—	—
Lister tous les utilisateurs du cloud	oui	oui	—	—	—	—
Lister les utilisateurs de son domaine	oui	oui	oui	oui	oui	—
Créer un utilisateur dans son domaine	oui	—	oui	oui	—	—
Créer un projet dans son domaine	oui	—	oui	oui	—	—
Attribuer member ou reader dans son domaine	oui	—	oui	oui	—	—
Attribuer le rôle admin	oui	—	oui	non	—	—
Consulter son propre compte	oui	oui	oui	oui	oui	oui
Changer son mot de passe	oui	oui	oui	oui	oui	oui
Créer un identifiant d'application pour soi	oui	oui	oui	oui	oui	oui

Le profil à retenir : manager de domaine. Il permet de déléguer toute la gestion courante d'un locataire sans donner le moyen de s'élever au rang d'administrateur.

Tableau de synthèse, construit à partir de la description des profils par défaut ; pour une opération précise, la règle effective fait foi (`oslopolicy-policy-generator`).

Chaque service a ses politiques

Service	Un `reader` du projet peut	Un `member` du projet peut en plus	Réservé à l'administration
Nova	**lister et consulter les instances**	créer, démarrer, arrêter, redimensionner, supprimer	migrer, évacuer, gérer hôtes, agrégats et gabarits
Neutron	**consulter réseaux, ports, routeurs**	créer réseaux, sous-réseaux, routeurs, groupes de sécurité	réseaux fournisseurs, réseaux partagés, agents
Cinder	**consulter volumes et instantanés**	créer, attacher, étendre, supprimer	types de volumes, quotas, migration de volumes
Glance	**lister et télécharger les images visibles**	téléverser des images privées, les partager	publier une image pour tout le cloud
Keystone	**voir son projet, ses limites**	rien de plus dans Keystone	identités, catalogue, domaines

- **Les mêmes quatre rôles partout :** c'est ce qui rend l'ensemble prévisible. Un utilisateur `member` d'un projet y est membre dans tous les services.
- **Le rôle `service` :** certaines opérations ne sont ouvertes qu'aux autres services — par exemple, Neutron signalant à Nova qu'un port est prêt.
- **Rôles propres à un service :** l'équilibrage de charge et l'orchestration, entre autres, définissent des rôles supplémentaires à attribuer explicitement.

```
# Lire une règle d'un autre service
oslopolicy-policy-generator --namespace nova \
  | grep 'servers:delete'

# Namespaces courants
# keystone · nova · neutron · cinder · glance · placement
```

> **Tableau indicatif** des règles par défaut récentes ; les détails varient selon la version et les options `[oslo_policy]` de chaque service.

Bonnes pratiques

Des groupes, pas des personnes

Toute attribution vise un groupe. Les arrivées et départs se règlent dans l'annuaire, pas dans OpenStack.

Le moindre privilège

`reader` par défaut, `member` pour ceux qui déploient, `manager` pour déléguer. `admin` reste l'exception.

Très peu d'administrateurs

Le rôle `admin` déborde encore de sa portée dans plusieurs services : le traiter comme un accès au cloud entier.

Les rôles par défaut

Éviter les rôles maison : aucun service ne les comprend sans surcharge de politique.

Pas de surcharge sans nécessité

Chaque règle modifiée est à revalider à chaque montée de version, service par service.

Des identifiants d'application

Pour tout ce qui n'est pas humain : limités à un projet, à un rôle, à une durée, révocables un par un.

Un compte de secours

Local, hors annuaire et hors fédération, mot de passe au coffre, usage tracé.

Une revue périodique

Extraire les attributions effectives et les faire valider par les responsables de projets.

```
# Qui est administrateur, où que ce soit ?
openstack role assignment list --names --role admin

# Tout ce que détient un groupe
openstack role assignment list --names \
  --group dev --group-domain entreprise --effective

# Les identifiants d'application d'un utilisateur
openstack application credential list --user alice \
  --user-domain nord
```

Erreur fréquente	Conséquence
donner `admin` « sur son projet »	droits bien plus larges que prévu
attribuer des rôles à des personnes	droits orphelins après un départ
partager le compte `admin` initial	aucune traçabilité
mot de passe dans un script	fuite ; rotation impossible sans tout casser
correspondance fédérée trop large	tout l'annuaire a accès au cloud

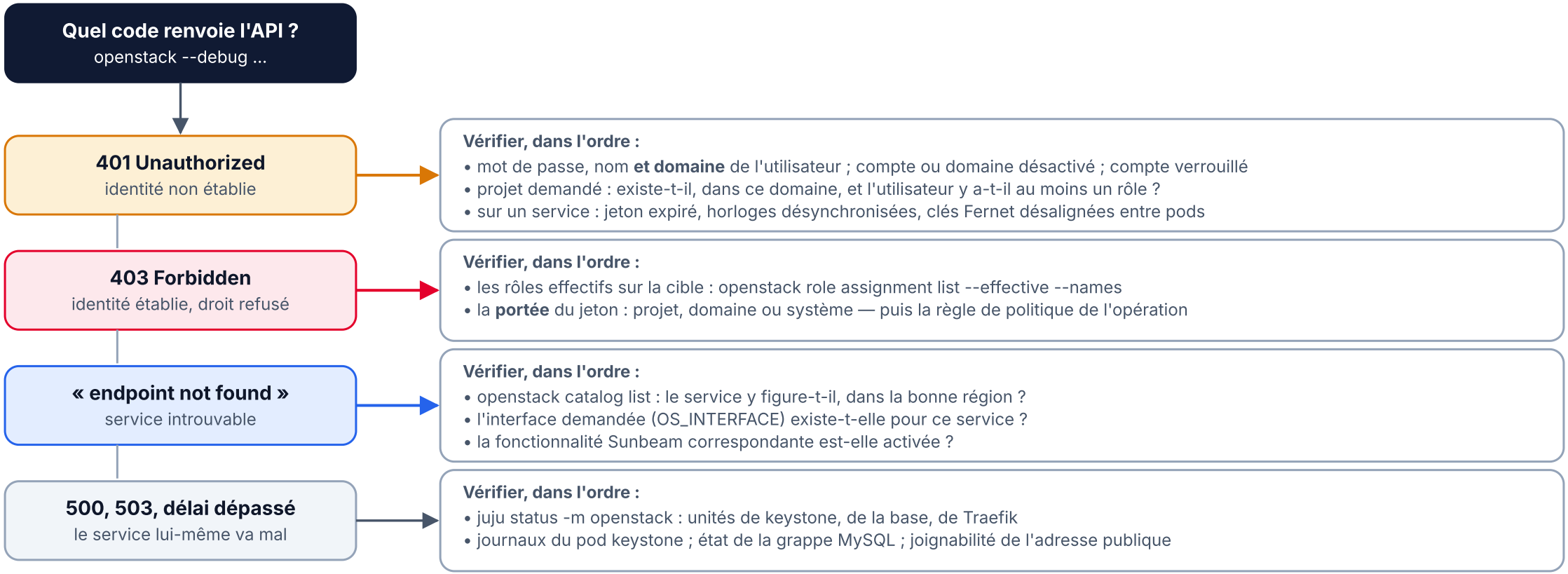
07

Exploiter Keystone sur Sunbeam

Savoir où regarder quand une authentification échoue, et ce qu'il faut protéger pour que cela n'arrive pas.

- Diagnostiquer un refus
- Disponibilité, sécurité, traçabilité
- Où regarder sur Sunbeam
- Synthèse

Diagnostiquer un refus



Où regarder sur Sunbeam

```
# 1 · L'application et ses relations
juju status -m openstack keystone --relations

# 2 · Les pods
sudo k8s kubectl -n openstack get pods | grep keystone

# 3 · Les journaux de Keystone
sudo k8s kubectl -n openstack logs keystone-0 \
  -c keystone --tail 100

# 4 · Les journaux du charm
juju debug-log -m openstack --include keystone/0

# 5 · La configuration réellement appliquée
sudo k8s kubectl -n openstack exec keystone-0 \
  -c keystone -- cat /etc/keystone/keystone.conf

# 6 · Les clés Fernet, pod par pod : mêmes fichiers ?
sudo k8s kubectl -n openstack exec keystone-0 \
  -c keystone -- ls /etc/keystone/fernet-keys
```

Constat	Piste
unité `blocked` ou `waiting`	lire le message d'état : relation manquante, base indisponible
401 intermittents	un pod n'a pas les mêmes clés que les autres ; horloge d'un nœud décalée
authentification lente	base de données ; annuaire LDAP lent à répondre
échec pour un seul domaine	configuration LDAP de ce domaine ; certificat de l'annuaire
connexion fédérée refusée	règles de correspondance ; URL de retour déclarée chez le fournisseur
tout le cloud en 401	Keystone injoignable par l'adresse interne ; Traefik interne

- **Journal détaillé :** l'option de débogage du charm, à activer le temps du diagnostic puis à retirer — les journaux détaillés sont volumineux.
- **Test de bout en bout le plus simple :** `openstack token issue`, d'abord depuis un nœud du cloud, puis depuis un poste extérieur.

> Noms des pods, des conteneurs et chemins à confirmer sur votre déploiement : ils dépendent de la révision des charms.

7 · EXPLOITATION

Disponibilité, sécurité, traçabilité

Sujet	Ce qui est en place sur un cloud Sunbeam à trois nœuds de contrôle	Ce qui reste à votre charge
Disponibilité de l'API	**plusieurs pods Keystone derrière Traefik ; aucun état local hormis les clés**	surveiller le nombre d'unités actives ; tester la perte d'un nœud
Base de données	**grappe MySQL répliquée**	sauvegarder : la base contient toutes les identités et attributions
Clés Fernet	**créées, tournées et distribuées par le charm**	restreindre l'accès à Juju et à Kubernetes : qui lit les clés fabrique des jetons
Chiffrement des échanges	**TLS sur les points d'accès par la fonctionnalité dédiée**	l'activer avant d'ouvrir le cloud ; distribuer l'autorité aux clients
Comptes de service	**créés et tournés par les relations entre charms**	ne jamais s'en servir pour des personnes
Compte administrateur	**créé à l'installation**	le réserver au secours ; comptes nominatifs pour l'exploitation
Horloges	—	synchronisation de tous les nœuds : un décalage fausse l'expiration des jetons
Traçabilité	**Keystone sait émettre des événements d'audit : authentifications, créations, attributions**	les collecter et les conserver ; activer l'audit des appels d'API si exigé

Trois actifs à protéger en priorité : la base de Keystone, le dépôt de clés Fernet, le compte administrateur. La compromission d'un seul donne la main sur tout le cloud.

Après une restauration de la base : les jetons en circulation restent valides — ils dépendent des clés, pas de la base — mais des attributions récentes peuvent avoir disparu : revérifier les droits.

SYNTHÈSE

Ce qu'il faut retenir

Entités et catalogue

- Acteur + rôle + cible : la seule source de droits.
- Le domaine délimite noms et délégation.
- Trois portées : projet, domaine, système.
- Le catalogue donne l'adresse de chaque service, par interface et par région.

Jetons

- Une preuve datée : identité et portée.
- Fernet : chiffré, non stocké, validé par Keystone.
- Trois sortes de clés ; rotation et distribution indissociables.
- Le jeton de service couvre les opérations longues.

Sources et politiques

- Local, LDAP, fédération — par domaine.
- Rôles et projets restent toujours dans Keystone.
- Chaque service décide, avec ses propres règles.
- 401 : qui êtes-vous ? 403 : pas le droit.

Sur Sunbeam, pour…	La voie à suivre
obtenir des identifiants d'administration	`sunbeam openrc`
raccorder un annuaire	`sunbeam enable ldap` **puis** `sunbeam ldap add-domain`
raccorder un fournisseur OIDC ou SAML	`sunbeam identity provider add` **, puis domaine, fournisseur, correspondance et protocole dans Keystone**
modifier le catalogue ou la configuration	par Sunbeam et les options du charm — jamais dans le pod
diagnostiquer	`openstack --debug` , `juju status` , journaux du pod `keystone`

Sources : documentation de Keystone (jetons, Fernet, protection des API, politiques), documentation de Canonical OpenStack 2026.1 (intégration LDAP, fournisseurs d'identité) — consultées en octobre 2026.